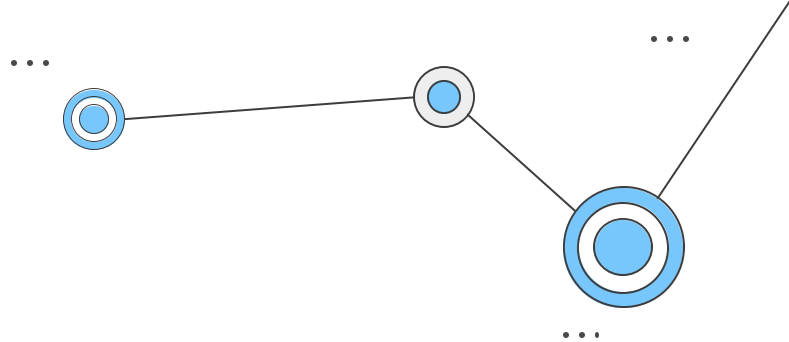
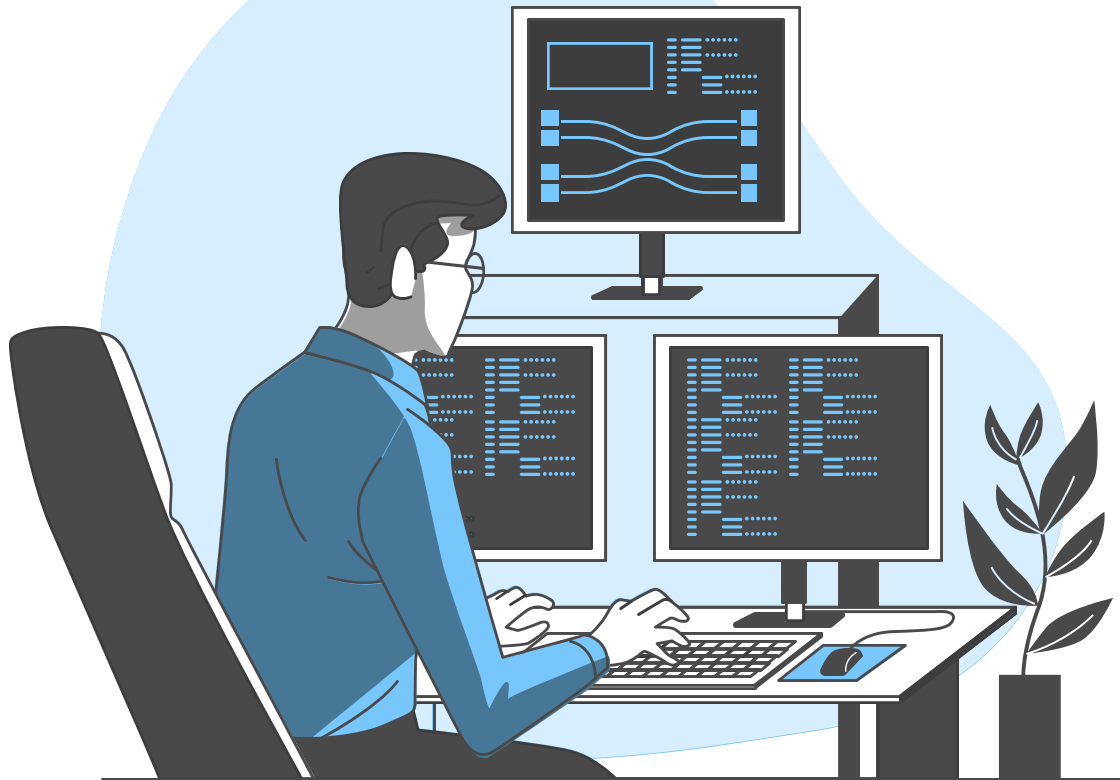




PUC Minas



Laboratório de Desenvolvimento de Software

Prof. Dr. João Paulo Aramuni

Sumário

- Construção de serviços Web
 - Principais frameworks
 - Organização das rotas

Sumário

- Construção de serviços Web
 - **Principais frameworks**
 - Organização das rotas

Construção de serviços Web

- Principais frameworks
 - Na aula anterior (*AULA 01_Arquitetura de microsserviços*), construímos nossa primeira **API REST** utilizando o **Spring Framework** e a extensão **Spring Boot** para auxiliar na configuração inicial do projeto.
 - <https://spring.io/guides/gs/rest-service/>



Construção de serviços Web

- Principais frameworks
 - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
 - <https://www.django-rest-framework.org/>



Construção de serviços Web

- Principais frameworks
 - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
 - <https://flask-restful.readthedocs.io/en/latest/>



FlaskRESTful



Construção de serviços Web

- Principais frameworks
 - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
 - <https://expressjs.com/en/5x/api.html>
 - <https://express-rest-api-generator.readthedocs.io/en/latest/>

express



Construção de serviços Web

- Principais frameworks
 - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
 - <https://go.dev/doc/tutorial/web-service-gin>



Construção de serviços Web

- Principais frameworks
 - Existem também outros frameworks que você pode experimentar para criar suas APIs REST, como por exemplo:
 - <https://dotnet.microsoft.com/pt-br/apps/aspnet/apis>

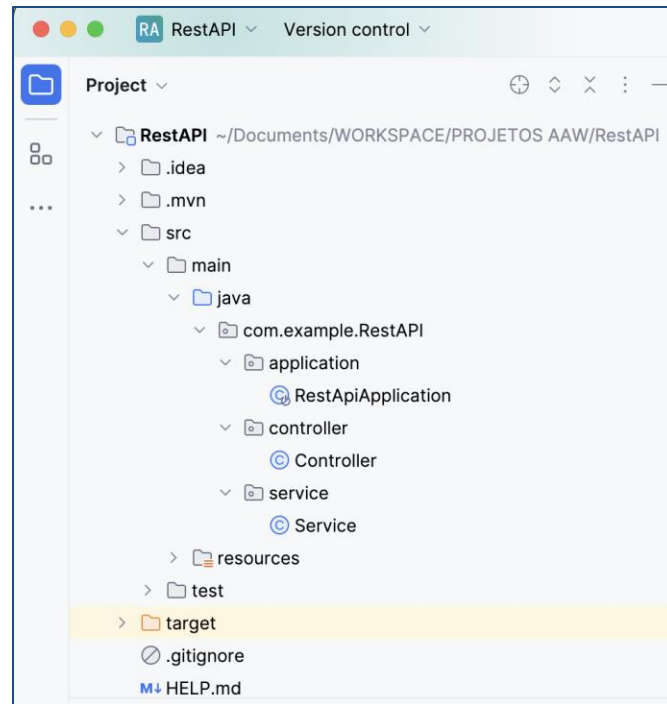
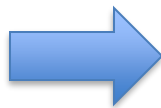
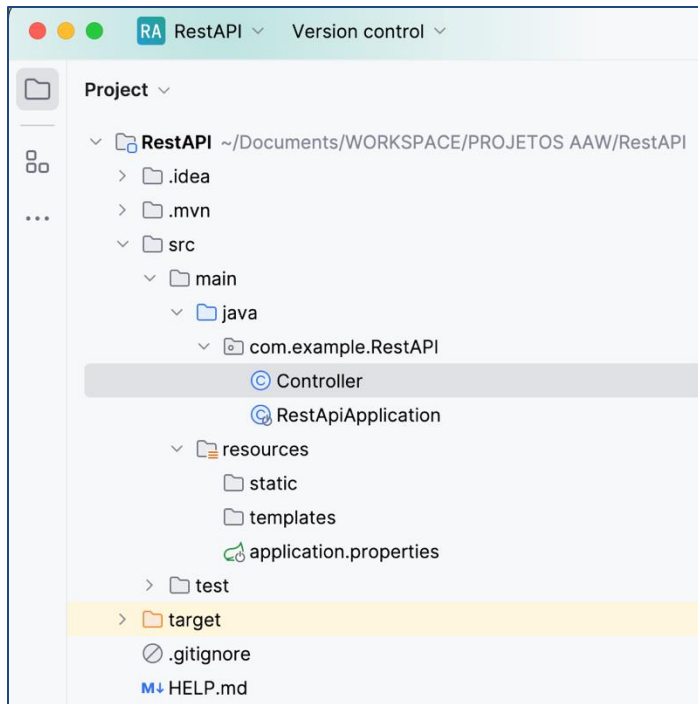


Sumário

- Construção de serviços Web
 - Principais frameworks
 - **Organização das rotas**

Organização das rotas

- Vamos agora aprimorar a arquitetura do nosso primeiro projeto de API REST e organizar melhor nossos **pacotes, serviços e rotas**.



Construção de serviços Web

- Organização das rotas
 - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
 - I) application
 - II) controller
 - III) service
 - Essa organização em camadas ou pacotes tem o objetivo de melhorar a modularidade, a manutenção e a compreensão do código.

Construção de serviços Web

- Organização das rotas
 - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
 - **I) application**
 - O pacote application geralmente contém a classe principal (Main) que inicia a aplicação.
 - Também pode incluir configurações de inicialização, configurações gerais do aplicativo e outros componentes relacionados à inicialização e configuração global.

Construção de serviços Web

- Organização das rotas
 - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
 - **II) controller**
 - O pacote controller é frequentemente usado para armazenar classes que atuam como controladores na arquitetura de uma aplicação.
 - Os controladores são responsáveis por receber as solicitações HTTP, interagir com os serviços subjacentes e retornar as respostas adequadas.

Construção de serviços Web

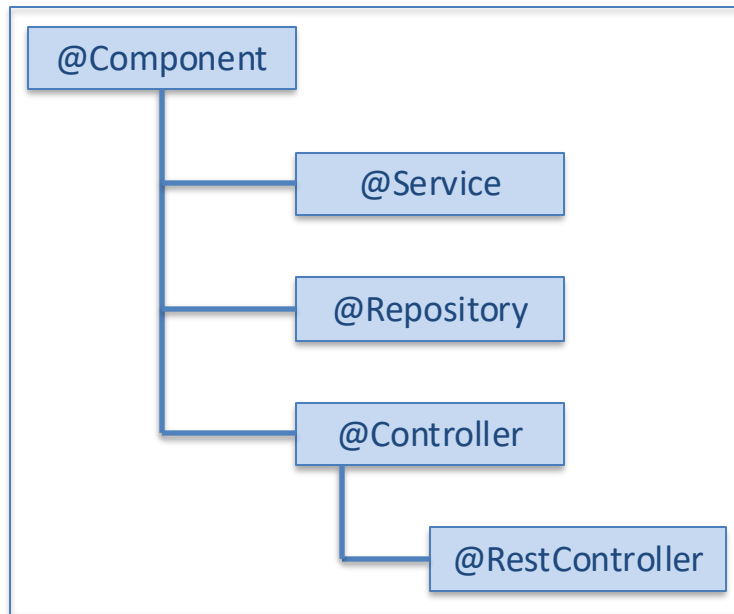
- Organização das rotas
 - Primeiramente, dividiremos a aplicação em três **pacotes** (packages):
 - **III) service**
 - O pacote service é destinado a conter as classes que implementam a lógica de negócio ou serviços.
 - Essas classes são responsáveis por manipular regras de negócio, realizar operações de processamento e interagir com o banco de dados, se necessário.

Construção de serviços Web

- Organização das rotas
 - Essa organização facilita a **manutenção**, a **escalabilidade** e a compreensão do código, pois cria uma separação clara de responsabilidades entre diferentes partes da aplicação. Ao seguir essa prática, a estrutura do projeto torna-se mais **modular** e **flexível**, facilitando a adição, remoção ou modificação de funcionalidades sem afetar outras partes da aplicação.
 - Essa abordagem também contribui para a **legibilidade do código**, pois permite que os desenvolvedores localizem rapidamente a funcionalidade relevante em seus respectivos pacotes.

Organização das rotas

- Futuramente, quando inserirmos um banco de dados na aplicação, teremos um quarto pacote (package), chamado **persistence**, **repository** ou **DAO** (Data Access Object).

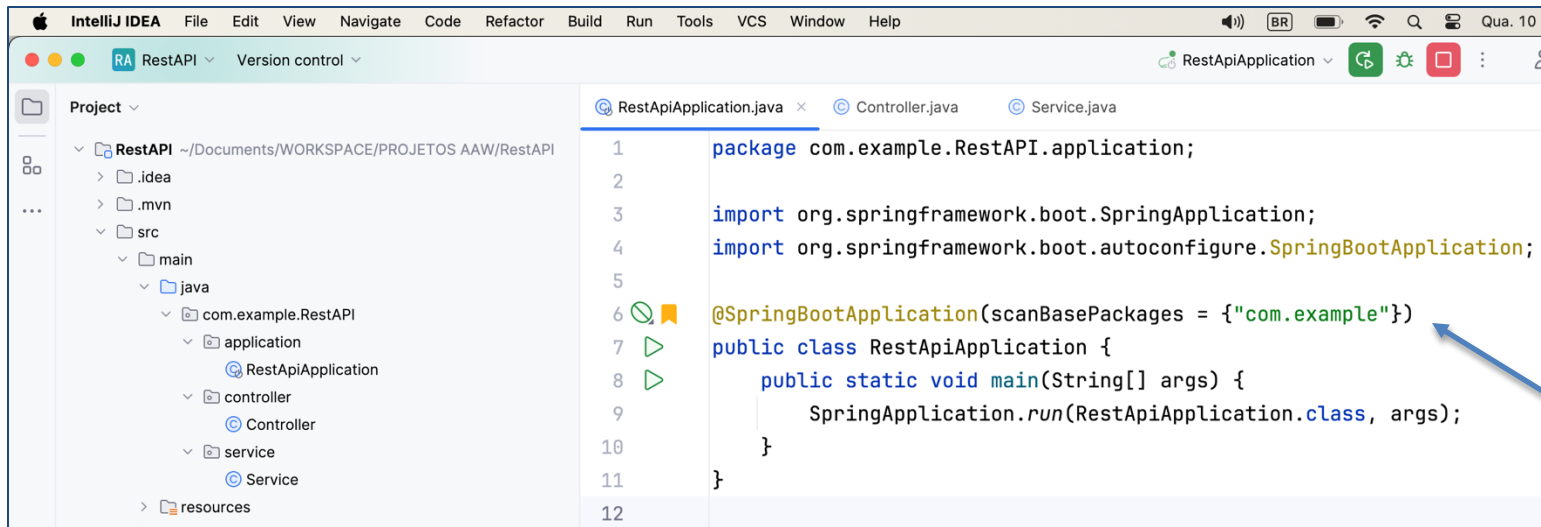


Construção de serviços Web

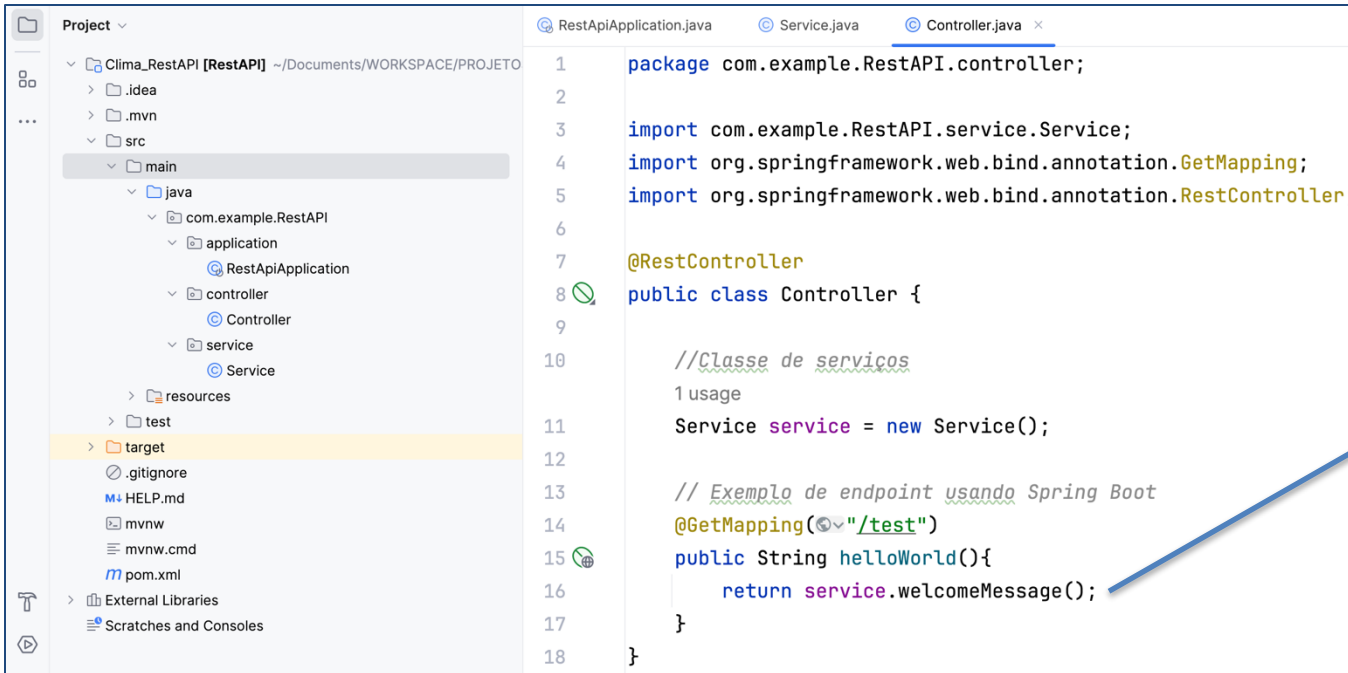
- Organização das rotas
 - A organização dos pacotes e a estrutura das rotas devem ser coerentes para proporcionar uma experiência de desenvolvimento mais consistente e facilitar a manutenção do código.
 - Quando um desenvolvedor trabalha em uma determinada funcionalidade, ele deve ser capaz de localizar facilmente os controladores associados e as rotas relacionadas nos pacotes apropriados. Isso melhora a legibilidade do código e a manutenção da aplicação.

Construção de serviços Web

Inclua o elemento: `(scanBasePackages = {"com.example"})` à frente da annotation: `@SpringBootApplication` para que os nossos novos pacotes sejam encontrados.

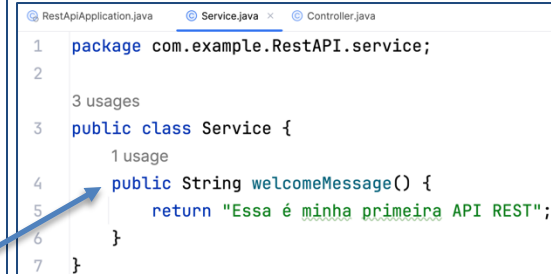


Altere a classe **Controller** para exportar a lógica de negócio para uma classe **Service**. Assim o controlador fica responsável apenas por criar as rotas (/test), enquanto as regras de negócio ficarão em uma classe de serviços.



The screenshot shows an IDE with a project named 'Clima_RestAPI [RestAPI]'. The project structure on the left includes 'main' (with 'java' containing 'com.example.RestAPI' which has 'application', 'controller', and 'service' sub-packages) and 'test'. The 'Controller.java' file is open in the editor, showing the following code:

```
1 package com.example.RestAPI.controller;
2
3 import com.example.RestAPI.service.Service;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController
8 public class Controller {
9
10     //Classe de serviços
11     1 usage
12     Service service = new Service();
13
14     // Exemplo de endpoint usando Spring Boot
15     @GetMapping("/test")
16     public String helloWorld(){
17         return service.welcomeMessage();
18     }
19 }
```



An inset screenshot shows the 'Service.java' file, which contains the business logic. The code is as follows:

```
1 package com.example.RestAPI.service;
2
3 public class Service {
4     1 usage
5     public String welcomeMessage() {
6         return "Essa é minha primeira API REST";
7     }
8 }
```

A blue arrow points from the `service.welcomeMessage()` call in the `Controller.java` file to the `welcomeMessage()` method in the `Service.java` file.

Construção de serviços Web

- Organização das rotas
 - Agora que já organizamos nossos **pacotes** e **classes**, vamos organizar nossas **rotas**.
 - Neste primeiro projeto de API REST, temos apenas uma rota definida.



```
// Exemplo de endpoint usando Spring Boot  
@GetMapping("/test")  
public String helloWorld(){  
    return service.welcomeMessage();  
}
```

Construção de serviços Web

- Organização das rotas
 - A importação `import org.springframework.web.bind.annotation.GetMapping;` refere-se a uma anotação específica do Spring Framework usada para mapear métodos de controle (controllers) a endpoints HTTP específicos.
 - Essa anotação faz parte do módulo spring-web e é usada principalmente para criar APIs RESTful.

Construção de serviços Web

- Organização das rotas
 - Organizar rotas em uma API REST é uma prática importante para manter o código claro, **modular** e fácil de entender.
 - Existem várias abordagens para organizar rotas, e a escolha pode depender do tamanho e complexidade do seu projeto.

Construção de serviços Web

- Organização das rotas

Aqui estão algumas estratégias comuns para organização de rotas:

- **Organização por Recurso:**

- Agrupe suas rotas com base nos recursos que a API fornece.
- Cada recurso (por exemplo: usuários, produtos, pedidos) terá seu próprio conjunto de rotas. Isso pode ser feito usando um padrão como `"/users"`, `"/products"`, `"/orders"`.
- Por hora, estamos utilizando esta abordagem para organizar nossas rotas.

Construção de serviços Web

- Organização das rotas

Aqui estão algumas estratégias comuns para organização de rotas:

- **Organização por Funcionalidade:**

- Agrupe suas rotas com base na funcionalidade que elas fornecem. Isso é útil quando você tem muitos recursos relacionados a uma funcionalidade específica.
- Exemplo: Autenticação, faturas, relatórios, etc.

Construção de serviços Web

- Organização das rotas

Aqui estão algumas estratégias comuns para organização de rotas:

- **Organização por Versão da API:**

- Se você espera que sua API evolua ao longo do tempo, pode ser útil versionar suas rotas (v1, v2, v3...) para garantir compatibilidade com versões anteriores.
 - /v1/users/
 - /v2/users/

Construção de serviços Web

- Organização das rotas

Aqui estão algumas estratégias comuns para organização de rotas:

- **Organização Hierárquica:**

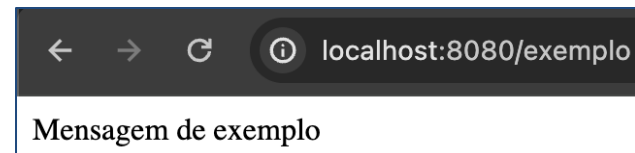
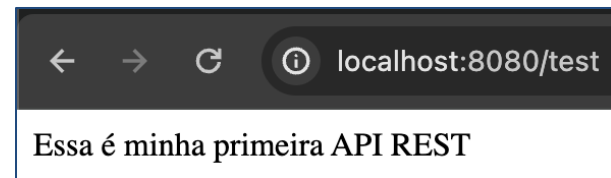
- Organize suas rotas de maneira hierárquica para representar relações de recursos aninhados.
 - /users
 - /users/{userId}
 - /users/{userId}/orders
 - /users/{userId}/orders/{orderId}

Construção de serviços Web

- Organização das rotas
 - Antes de avançarmos para o próximo exemplo prático, vamos criar mais uma rota de exemplo.
 - Teremos assim, duas rotas e dois serviços.


```
RestApiApplication.java  Service.java  Controller.java x
1 package com.example.RestAPI.controller;
2
3 import com.example.RestAPI.service.Service;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController
8 public class Controller {
9
10     //Classe de serviços
11     2 usages
12     Service service = new Service();
13
14     // Exemplo de endpoint usando Spring Boot
15     @GetMapping("/test")
16     public String helloWorld(){
17         return service.welcomeMessage();
18     }
19
20     @GetMapping("/exemplo")
21     public String exemplo(){
22         return service.exampleMessage();
23     }
24 }
```

```
RestApiApplication.java  Service.java x  Controller.java
3 public class Service {
4     1 usage
5     public String welcomeMessage() {
6         return "Essa é minha primeira API REST";
7     }
8     1 usage
9     public String exampleMessage() {
10         return "Mensagem de exemplo";
11     }
12 }
```



Vamos à prática!

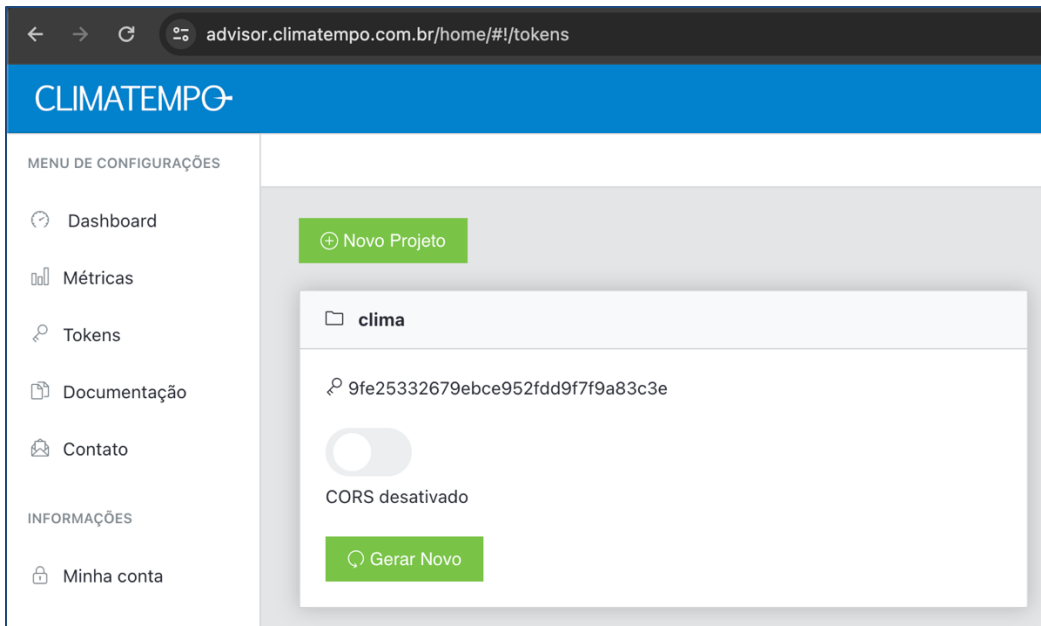
- Crie uma API REST para consultar o clima em BH.
- Para consultar as informações sobre o clima, utilizaremos a API CLIMATEMPO.
 - Para verificar o clima de uma cidade apenas, a API é gratuita.
 - <https://advisor.climatempo.com.br/>

Vamos à prática!



Vamos à prática!

- Para utilizar a API CLIMATEMPO é necessário cadastrar um email, logar e gerar um token:



Vamos à prática!

- Link para a documentação que utilizaremos como guia:
- <https://apiadvisor.climatepo.com.br/doc/index.html#api-Climate-ClimateTemperatureByCityOrLatLon>

The screenshot shows a web browser displaying the API documentation for 'Climate - Climate temperature'. The page has a dark theme. On the left, there is a sidebar with a search bar and a list of API endpoints. The 'Climate temperature' endpoint is selected and highlighted in blue. The main content area shows the endpoint details, including the HTTP method (GET), the endpoint URL, an example request, and a table of parameters.

Climate - Climate temperature. 1.0.0

Climate temperature by city's Id or latitude and longitude.

GET

`/api/v1/climate/temperature/locale/:id?token=your-app-token`

Exemplo:

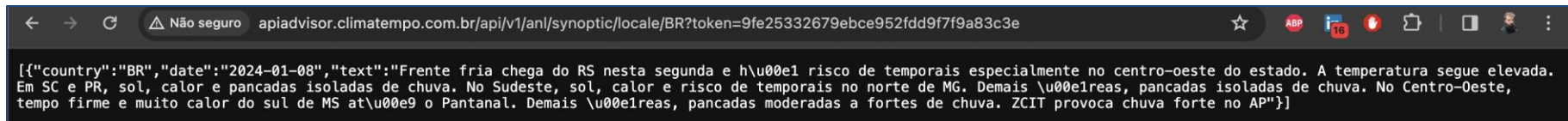
`curl -v -L "http://apiadvisor.climatepo.com.br/api/v1/climate/temperature/locale/3477?token=your-app-token"`

Parameter

Field	Type	Description
id	Number	City's Id. Search ID.
latitude	Number	Spot latitude (Optional).
longitude	Number	Spot longitude (Optional).
token	String	API access token.

Vamos à prática!

- Antes de partir para o código, faça um teste no browser de sua preferência para verificar se a chamada à API está funcionando:
 - <http://apiadvisor.climatempo.com.br/api/v1/anl/synoptic/locale/BR?token=9fe25332679ebce952fdd9f7f9a83c3e>
 - Para o nível Brasil está funcionando corretamente.



Vamos à prática!

- Já para o nível Belo Horizonte, será necessário cadastrar a cidade para aquele token gerado (lembrando que somente uma cidade é permitida no plano gratuito).
 - ID da cidade de Belo Horizonte: **6879**
 - <https://apiadvisor.climatempo.com.br/api/v1/locale/city?name=Belo%20Horizonte&state=MG&token=9fe25332679ebce952fdd9f7f9a83c3e>

← → ↻  apiadvisor.climatempo.com.br/api/v1/locale/city?name=Belo%20Horizonte&state=MG&token=9fe25332679ebce952fdd9f7f9a83c3e

```
[{"id":6879,"name":"Belo Horizonte","state":"MG","country":"BR"}]
```

Vamos à prática!

- Para referenciar o ID 6879 (BH) ao nosso token, será necessário rodar o seguinte comando:

UserTokenManagement - Register city's Id to token.

1.0.0 ▾

Register city's Id to token.

PUT

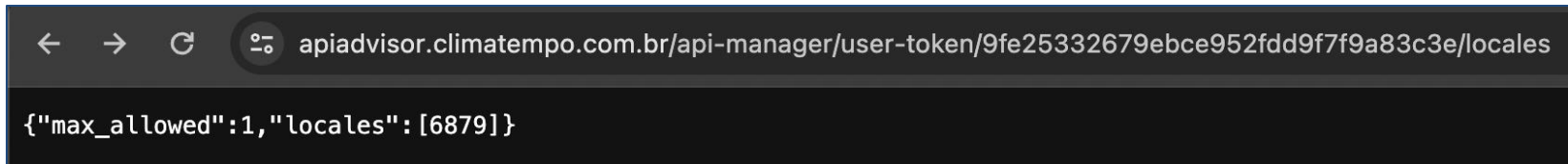
```
/api-manager/user-token/:token/locales
```

Example:

```
curl -X PUT \
  'http://apiadvisor.climatepo.com.br/api-manager/user-token/:your-app-token/locales' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'localeId[]=3477'
```


Vamos à prática!

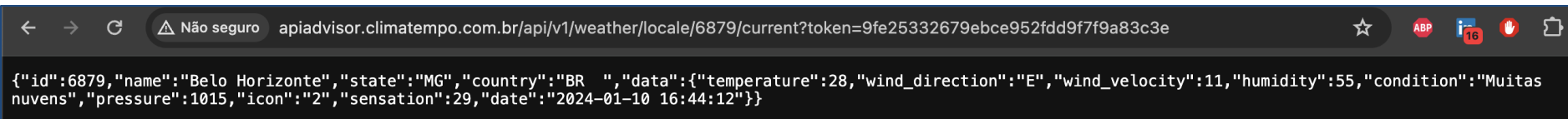
- Agora temos a cidade de BH cadastrada para o nosso token.
 - <https://apiadvisor.climatempo.com.br/api-manager/user-token/9fe25332679ebce952fdd9f7f9a83c3e/locales>
- Caso você não tenha interesse em criar um cadastro no CLIMATEMPO, ainda assim poderá utilizar as URLs desta aula normalmente.

A screenshot of a web browser's developer tools or REST client interface. The address bar shows the URL: https://apiadvisor.climatempo.com.br/api-manager/user-token/9fe25332679ebce952fdd9f7f9a83c3e/locales. Below the address bar, the response body is displayed as a JSON object: {"max_allowed":1,"locales":[6879]}.

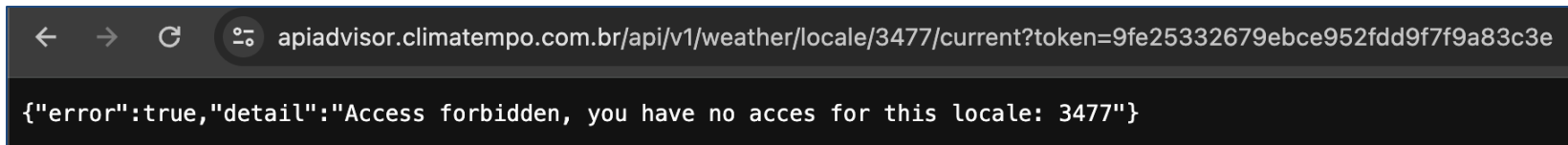
```
← → ↻ ⓘ apiadvisor.climatempo.com.br/api-manager/user-token/9fe25332679ebce952fdd9f7f9a83c3e/locales  
  
{"max_allowed":1,"locales":[6879]}
```

Vamos à prática!

- Agora temos as informações sobre o clima da cidade de Belo Horizonte aparecendo no browser (Google Chrome):
 - <https://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e>



- Caso não tivéssemos feito esse processo de registrar a cidade de BH para o token, a API retornaria a seguinte mensagem:

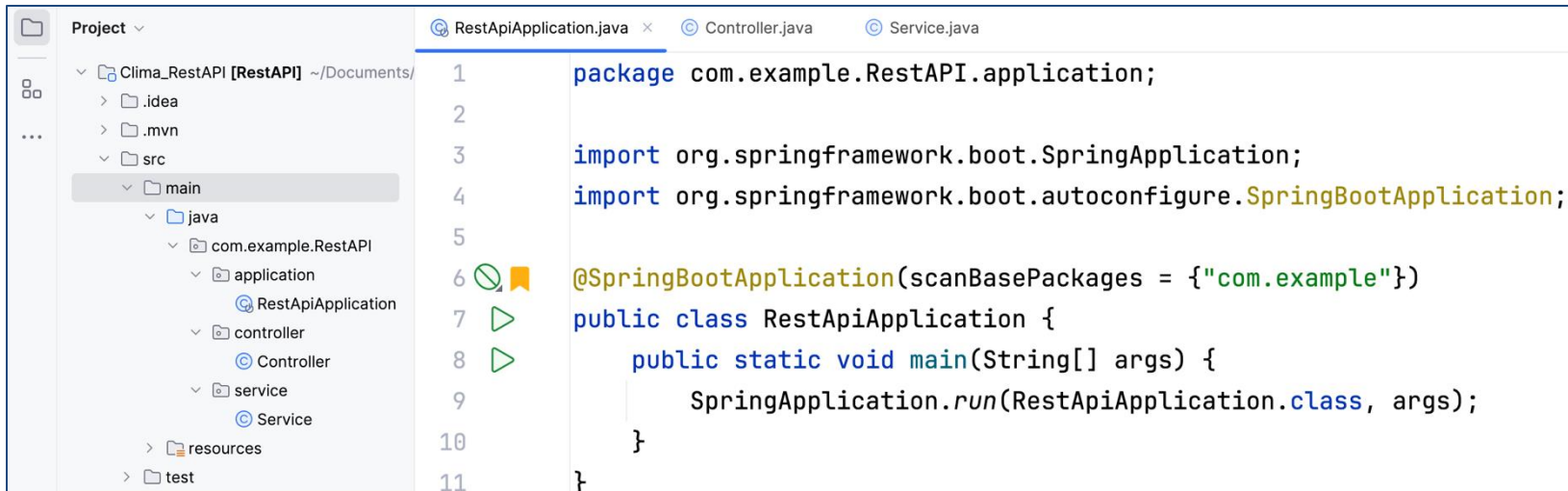


Vamos à prática!

- Uma vez que a chamada à API está funcionando no browser, podemos levar o link de BH para o nosso código Java.
- Faremos uma estrutura de projeto muito semelhante ao primeiro exemplo, com os pacotes **application**, **controller** e **service**.

Vamos à prática!

- Classe **RestApiApplication** com o método **main**:



The screenshot shows an IDE with a project named 'Clima_RestAPI [RestAPI]' located at '~/.Documents/'. The project structure is as follows:

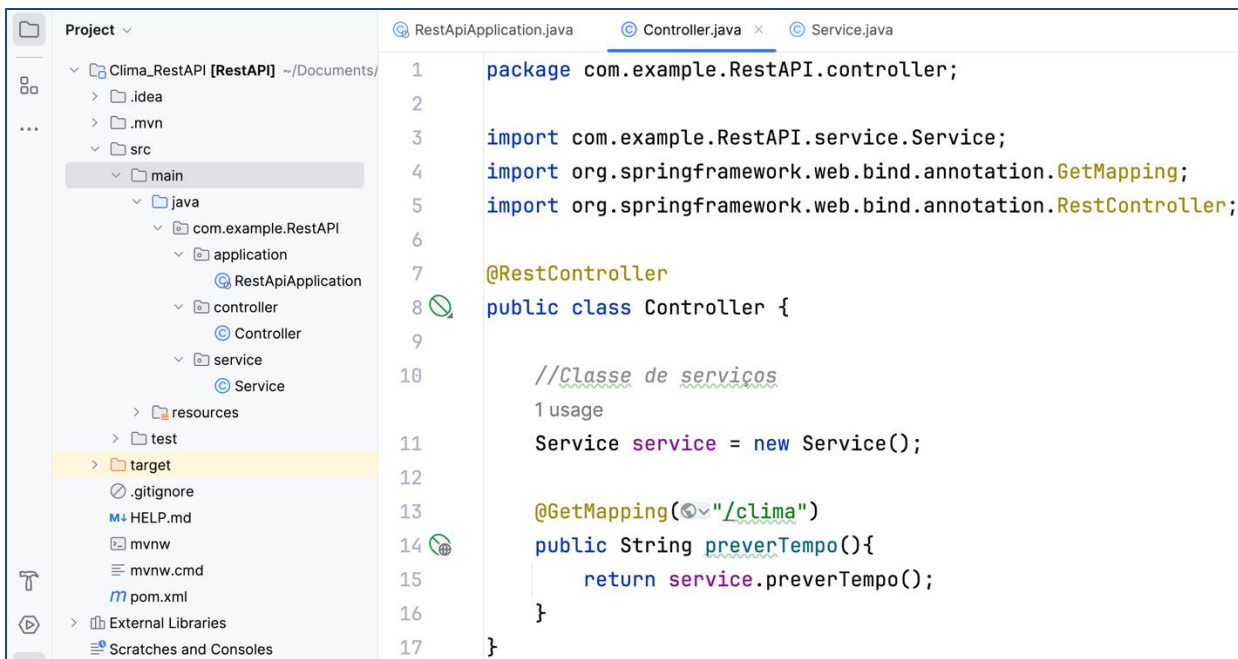
- src
 - main
 - java
 - com.example.RestAPI
 - application
 - RestApiApplication
 - controller
 - Controller
 - service
 - Service
 - resources
 - test

The code editor shows the following code for `RestApiApplication.java`:

```
1 package com.example.RestAPI.application;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication(scanBasePackages = {"com.example"})
7 public class RestApiApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(RestApiApplication.class, args);
10    }
11 }
```

Vamos à prática!

- Classe **Controller** com o **endpoint** e a chamada para a classe **Service**:



The screenshot shows an IDE with a project named 'Clima_RestAPI [RestAPI]' located at '~\Documents/'. The project structure is visible on the left, showing a 'main' directory with 'java' and 'resources' subdirectories. The 'java' directory contains a package 'com.example.RestAPI' with sub-packages 'application', 'controller', and 'service'. The 'controller' package contains a 'Controller' class. The 'service' package contains a 'Service' class. The 'resources' directory contains a 'target' directory. The 'target' directory is highlighted in yellow. The 'Controller' class is open in the editor, showing the following code:

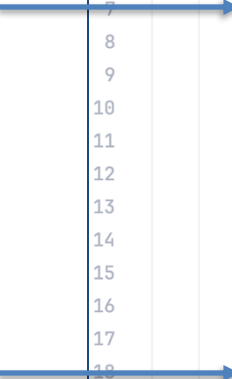
```
1 package com.example.RestAPI.controller;
2
3 import com.example.RestAPI.service.Service;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController
8 public class Controller {
9
10     //Classe de serviços
11     1 usage
12     Service service = new Service();
13
14     @GetMapping("/clima")
15     public String preverTempo(){
16         return service.preverTempo();
17     }
18 }
```

- Classe **Service** contendo o serviço de previsão do tempo:
 - Repare que o link para o clima de BH foi inserido na variável String **apiUrl**.



```
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 3 usages
6 public class Service {
7     1 usage
8     public String preverTempo() {
9         String dadosMeteorologicos = "";
10        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
11
12        RestTemplate restTemplate = new RestTemplate();
13        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
14
15        if (responseEntity.getStatusCode().is2xxSuccessful()) {
16            dadosMeteorologicos = responseEntity.getBody();
17        } else {
18            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
19        }
20        return dadosMeteorologicos;
21    }
22 }
```

- Aqui temos a variável que será retornada do **Service** para o **Controller**.
 - String **dadosMeteorologicos**.



```
RestApiApplication.java  Controller.java  Service.java x
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 3 usages
6 public class Service {
7     1 usage
8     public String preverTempo() {
9         String dadosMeteorologicos = "";
10        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
11
12        RestTemplate restTemplate = new RestTemplate();
13        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
14
15        if (responseEntity.getStatusCode().is2xxSuccessful()) {
16            dadosMeteorologicos = responseEntity.getBody();
17        } else {
18            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
19        }
20        return dadosMeteorologicos;
21    }
22 }
```

- Aqui estamos utilizando a classe **RestTemplate** do **Spring Framework** para fazer uma requisição **HTTP GET** para uma determinada URL (**apiUrl**) e receber a resposta como uma **String**.

```
RestApiApplication.java  Controller.java  Service.java x
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 3 usages
6 public class Service {
7     1 usage
8     public String preverTempo() {
9         String dadosMeteorologicos = "";
10        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
11
12        RestTemplate restTemplate = new RestTemplate();
13        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
14
15        if (responseEntity.getStatusCode().is2xxSuccessful()) {
16            dadosMeteorologicos = responseEntity.getBody();
17        } else {
18            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
19        }
20        return dadosMeteorologicos;
21    }
22 }
```


- Aqui estamos verificando se a requisição foi bem-sucedida. Ou seja, se está na faixa de códigos 2xx (Normalmente 200 = sucesso, 500 = erro interno do servidor, 404 = página não encontrada, etc.).

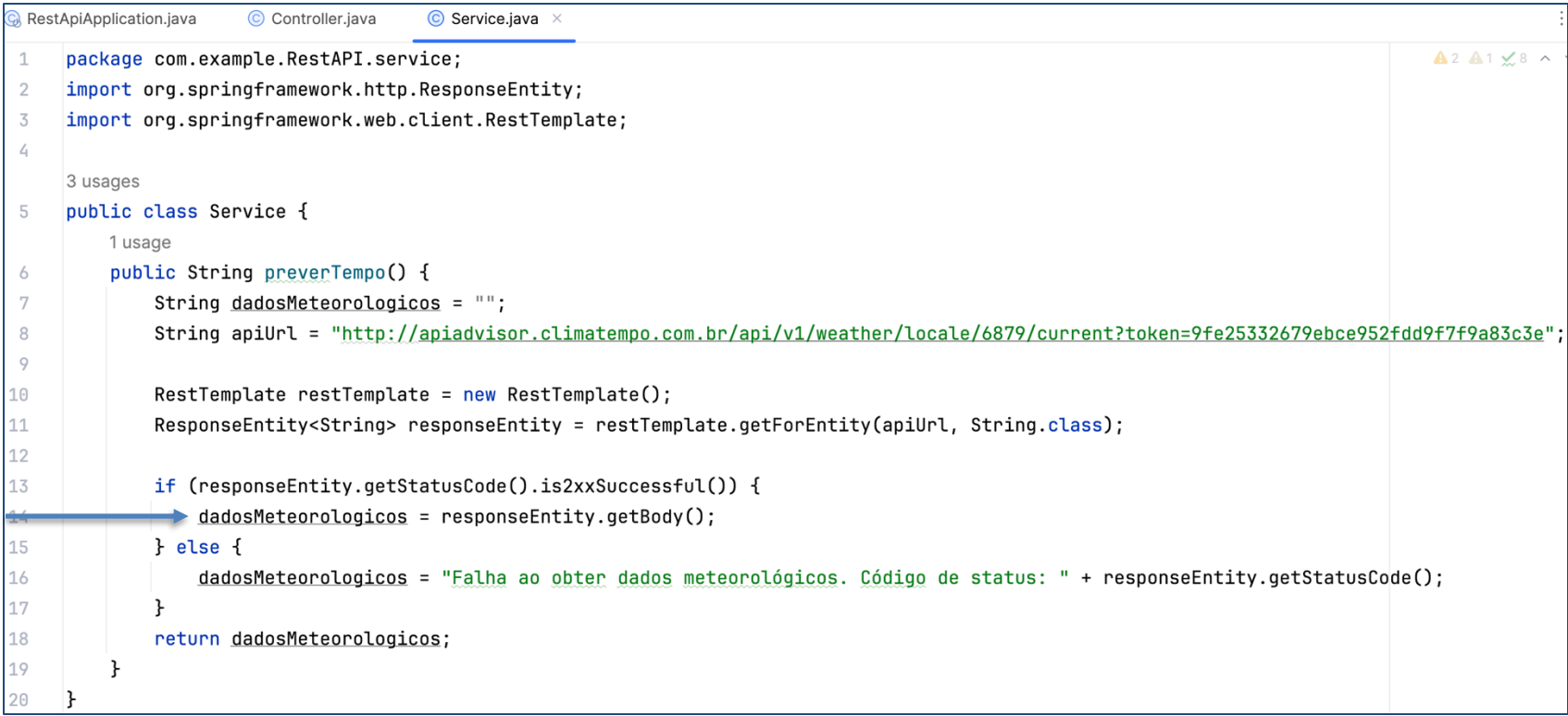


The screenshot shows an IDE with three tabs: RestApiApplication.java, Controller.java, and Service.java. The Service.java tab is active, displaying the following code:

```
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 public class Service {
6     public String preverTempo() {
7         String dadosMeteorologicos = "";
8         String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
9
10        RestTemplate restTemplate = new RestTemplate();
11        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
12
13        if (responseEntity.getStatusCode().is2xxSuccessful()) {
14            dadosMeteorologicos = responseEntity.getBody();
15        } else {
16            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
17        }
18        return dadosMeteorologicos;
19    }
20 }
```

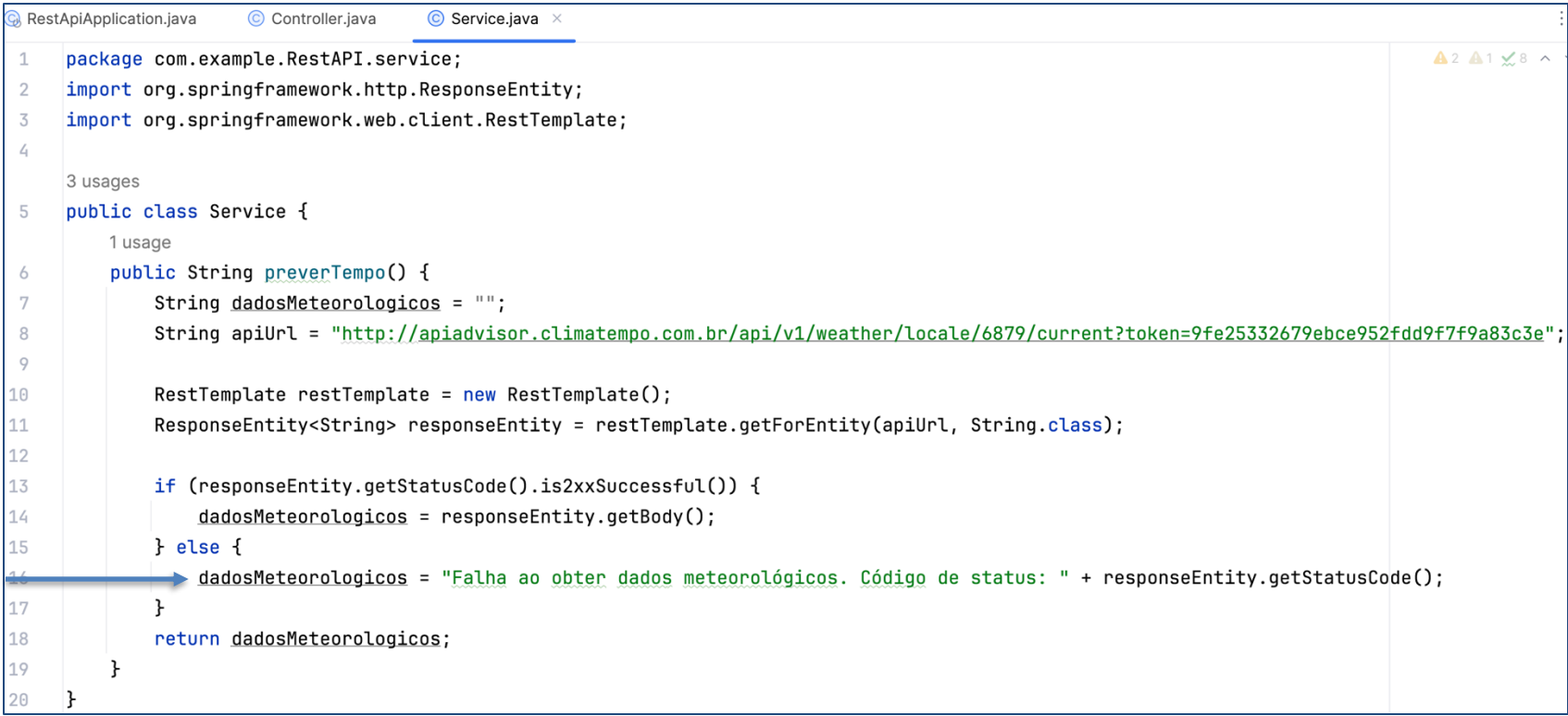
A blue arrow points to line 13, highlighting the `if (responseEntity.getStatusCode().is2xxSuccessful())` condition.

- Caso a requisição tenha sido bem-sucedida, guardamos o corpo de sua resposta.
 - Utilizamos para isso o método **getBody()** da classe **ResponseEntity**.



```
RestApiApplication.java Controller.java Service.java x
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 3 usages
6 public class Service {
7     1 usage
8     public String preverTempo() {
9         String dadosMeteorologicos = "";
10        String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
11
12        RestTemplate restTemplate = new RestTemplate();
13        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
14
15        if (responseEntity.getStatusCode().is2xxSuccessful()) {
16            dadosMeteorologicos = responseBody.getBody();
17        } else {
18            dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
19        }
20        return dadosMeteorologicos;
21    }
22 }
```

- Caso a requisição **não** tenha sido bem-sucedida, guardamos o código do status.
 - Utilizamos para isso o método **getStatusCode()** da classe **ResponseEntity**.



```
1 package com.example.RestAPI.service;
2 import org.springframework.http.ResponseEntity;
3 import org.springframework.web.client.RestTemplate;
4
5 public class Service {
6     public String preverTempo() {
7         String dadosMeteoroLogicos = "";
8         String apiUrl = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
9
10        RestTemplate restTemplate = new RestTemplate();
11        ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
12
13        if (responseEntity.getStatusCode().is2xxSuccessful()) {
14            dadosMeteoroLogicos = responseEntity.getBody();
15        } else {
16            dadosMeteoroLogicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
17        }
18        return dadosMeteoroLogicos;
19    }
20 }
```

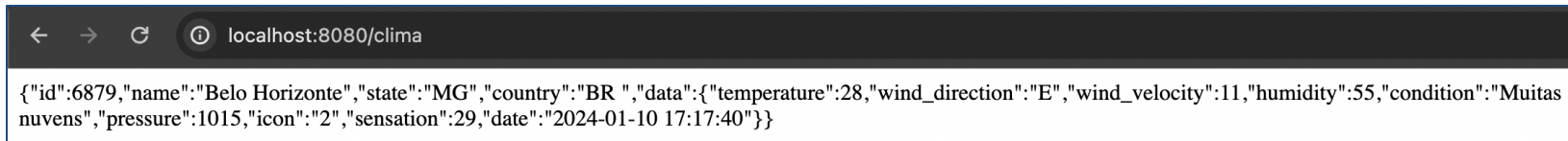
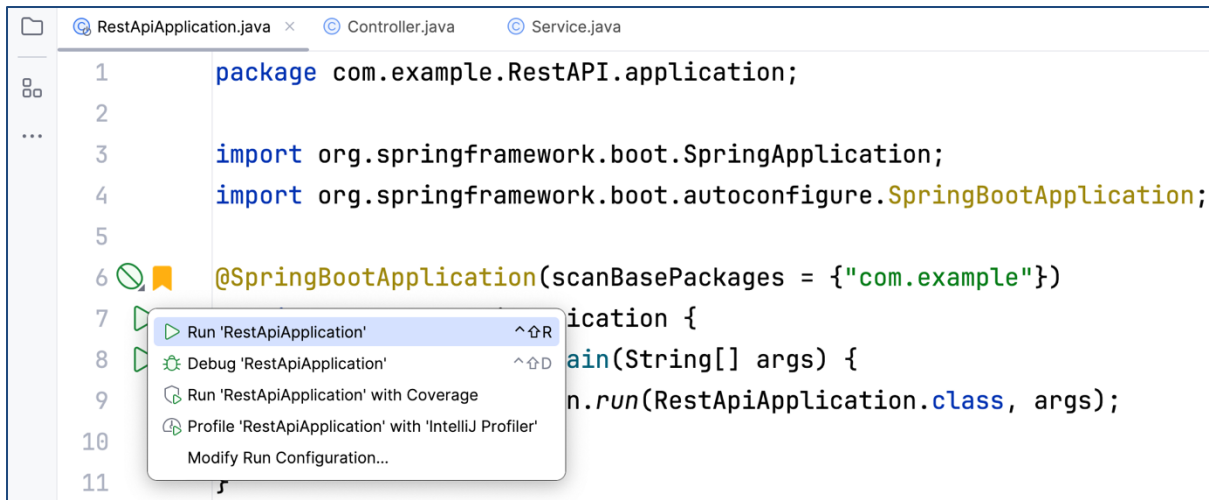
- Por último, retornamos o resultado do nosso serviço, seja ele o corpo da resposta ou o código de status, para a classe **Controller**.



```
RestApiApplication.java  Controller.java  Service.java x
1  package com.example.RestAPI.service;
2  import org.springframework.http.ResponseEntity;
3  import org.springframework.web.client.RestTemplate;
4
5  3 usages
6  public class Service {
7      1 usage
8      public String preverTempo() {
9          String dadosMeteorologicos = "";
10         String apiURL = "http://apiadvisor.climatempo.com.br/api/v1/weather/locale/6879/current?token=9fe25332679ebce952fdd9f7f9a83c3e";
11
12         RestTemplate restTemplate = new RestTemplate();
13         ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiURL, String.class);
14
15         if (responseEntity.getStatusCode().is2xxSuccessful()) {
16             dadosMeteorologicos = responseEntity.getBody();
17         } else {
18             dadosMeteorologicos = "Falha ao obter dados meteorológicos. Código de status: " + responseEntity.getStatusCode();
19         }
20         return dadosMeteorologicos;
21     }
22 }
```

Vamos à prática!

- Por fim, basta rodar a aplicação e ver o resultado, em formato **JSON**, no browser:



Bônus

- Até o momento, utilizamos o browser (**Google Chrome**) para enviar as requisições HTTP para a nossa API REST.
 - [GET] <http://localhost:8080/clima>
- Porém no mercado de trabalho é mais comum usarmos programas específicos para essa finalidade, como o **Postman** e o **Insomnia**.



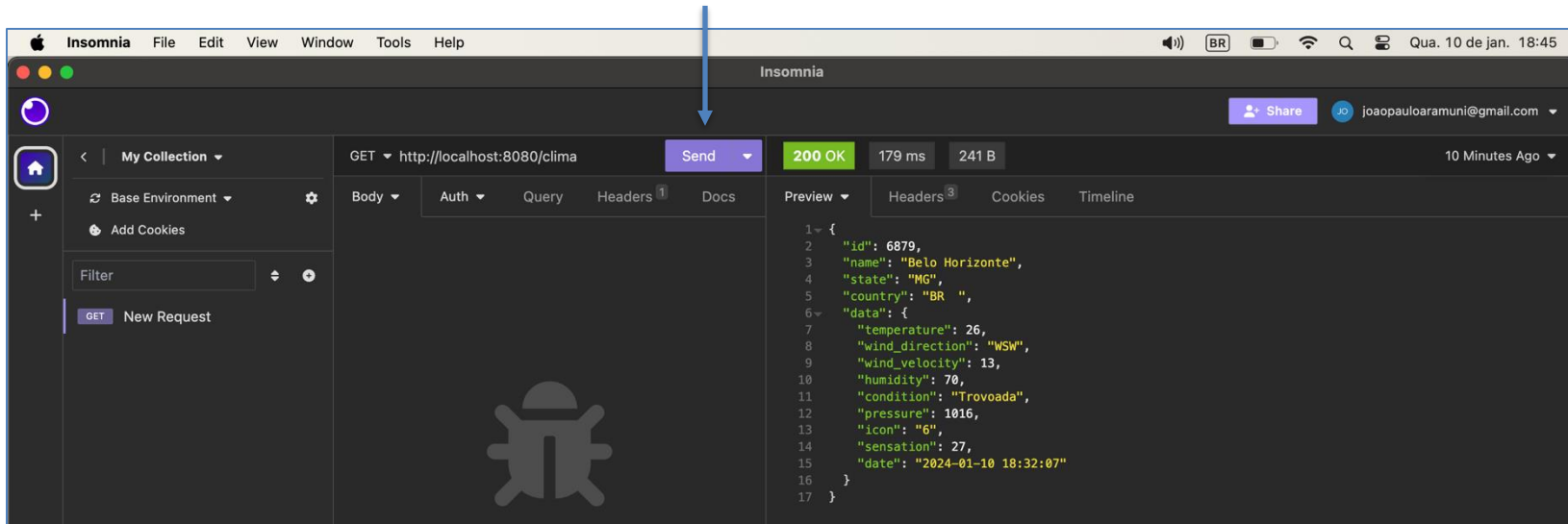
POSTMAN



Insomnia

Bônus

- Faça o download e teste você mesmo:
 - <https://www.postman.com/downloads/>
 - <https://insomnia.rest/download>



Exercício

- **FIPE API HTTP REST**
 - API de Consulta Tabela **FIPE**
- Crie uma **API REST** em Java, utilizando o Spring Framework, para consultar informações sobre marcas, modelos e valores de carros da tabela FIPE.
- Utilize a seguinte API para consumir as informações da tabela FIPE.
 - <https://deividfortuna.github.io/fipe/>
 - <https://parallelum.com.br/fipe/api/v1>

Exercício

- **FIPE API HTTP REST**

- A API de Consulta Tabela FIPE fornece preços médios de veículos no mercado nacional. Atualizada mensalmente com dados extraídos da tabela FIPE.
- Essa API Fipe utiliza banco de dados próprio, onde todas as requisições acontecem internamente, sem sobrecarregar o Web Service da Fipe, evitando assim bloqueios por múltiplos acessos.

Exercício

- **FIPE API HTTP REST**

- A API está online desde 2015 e é totalmente gratuita.
- Ela é mantida pelo Dev Deivid Fortuna: deividfortuna@gmail.com
- <https://github.com/deividfortuna/fipe>

FIPE API HTTP REST

Você pode usar a biblioteca em PHP desenvolvida para consumir a API <https://github.com/deividfortuna/fipe>.

A API está **online desde 2015 e totalmente gratuita**. O que acha de me pagar um cerveja? 🍺

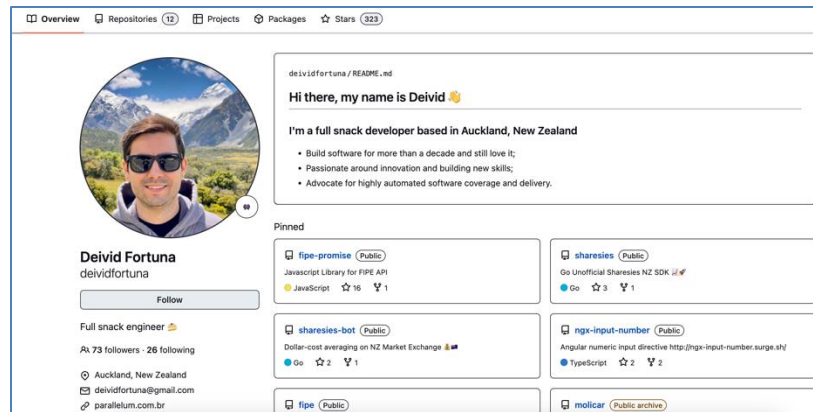


A API disponibiliza seus dados de busca no formato JSON. Confira a URL base de acesso a API:

<https://parallelum.com.br/fipe/api/v1>

A Tabela Fipe expressa preços médios de veículos no mercado nacional, servindo apenas como um parâmetro para negociações ou avaliações. Os preços efetivamente praticados variam em função da região, conservação, cor, acessórios ou qualquer outro fator que possa influenciar as condições de oferta e procura por um veículo específico.

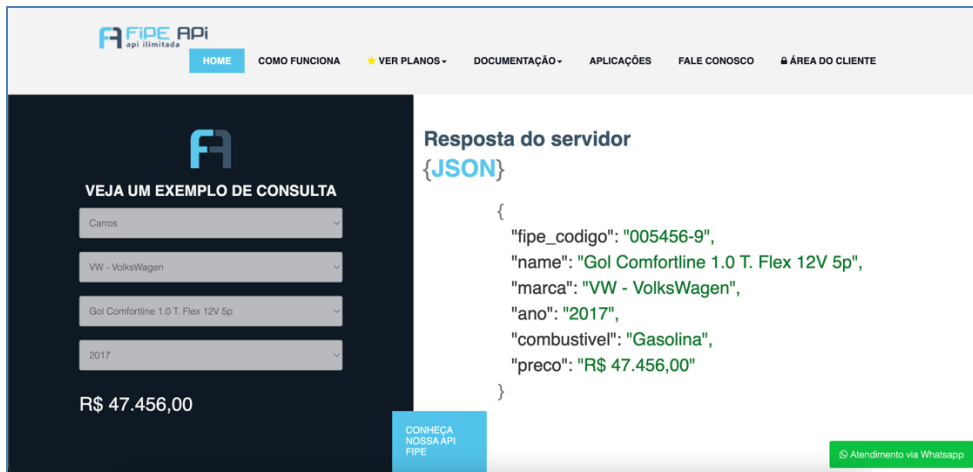
Essa API Fipe utiliza banco de dados próprio, onde todas as requisições acontecem internamente, sem sobrecarregar o Web Service da Fipe, evitando assim bloqueios por múltiplos acessos.



Exercício

- **FIPE API HTTP REST**

- Obs.: Existem outras APIs que também fornecem o mesmo serviço de consulta à Tabela FIPE, porém necessitam de cadastro, token e/ou são pagas.



The screenshot shows the FIPE API website interface. On the left, there's a section titled "VEJA UM EXEMPLO DE CONSULTA" with four dropdown menus containing the values: "Carnos", "VW - VolksWagen", "Gol Comfortline 1.0 T. Flex 12V 5p", and "2017". Below these, the price "R\$ 47.456,00" is displayed. On the right, under the heading "Resposta do servidor {JSON}", a JSON object is shown with the following fields: "fipe_codigo": "005456-9", "name": "Gol Comfortline 1.0 T. Flex 12V 5p", "marca": "VW - VolksWagen", "ano": "2017", "combustivel": "Gasolina", and "preco": "R\$ 47.456,00". At the bottom right, there's a green button labeled "Atendimento via Whatsapp".



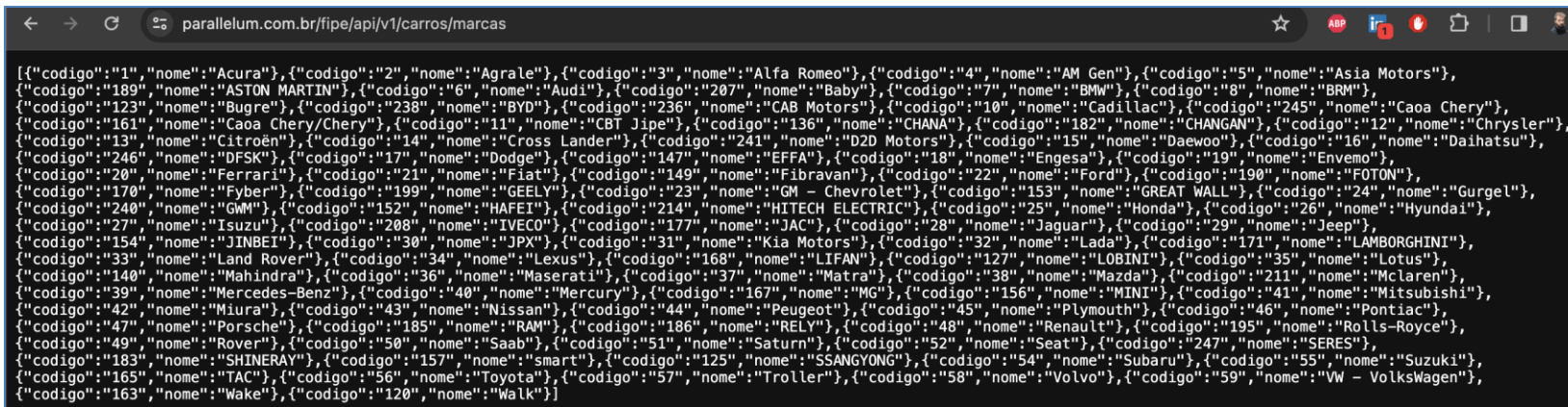
The screenshot shows the FIPE API website documentation page. The header includes the FIPE logo and navigation links: Home, Planos, Consulta, Documentação, Contato/FAQ, and Status. A red button labeled "Área do cliente" is in the top right. The main heading is "Documentação" with the subtitle "Confira a documentação do API". Below this, there's a section titled "Sobre" with the text: "A API Fipe expressa preços médios de veículos anunciados pelos vendedores, no mercado nacional, servindo como um parâmetro para negociações ou avaliações." Another section titled "Token" explains that the token is provided after payment of the plan and includes a note: "*Você pode obter um token de demonstração gratuito, basta criar sua conta no site e ir até o menu (Meus Planos) para visualizar o token."

Exercício

- **FIPE API HTTP REST**

- Antes de partir para o código Java, faça um teste das **URLs** no browser de sua preferência:

- <https://parallelum.com.br/fipe/api/v1/carros/marcas>



```
[{"codigo": "1", "nome": "Acura"}, {"codigo": "2", "nome": "Agrale"}, {"codigo": "3", "nome": "Alfa Romeo"}, {"codigo": "4", "nome": "AM Gen"}, {"codigo": "5", "nome": "Asia Motors"}, {"codigo": "189", "nome": "ASTON MARTIN"}, {"codigo": "6", "nome": "Audi"}, {"codigo": "207", "nome": "Baby"}, {"codigo": "7", "nome": "BMW"}, {"codigo": "8", "nome": "BRM"}, {"codigo": "123", "nome": "Bugre"}, {"codigo": "238", "nome": "BYD"}, {"codigo": "236", "nome": "CAB Motors"}, {"codigo": "10", "nome": "Cadillac"}, {"codigo": "245", "nome": "Caoa Chery"}, {"codigo": "161", "nome": "Caoa Chery/Chery"}, {"codigo": "11", "nome": "CBT Jipe"}, {"codigo": "136", "nome": "CHANA"}, {"codigo": "182", "nome": "CHANGAN"}, {"codigo": "12", "nome": "Chrysler"}, {"codigo": "13", "nome": "Citroën"}, {"codigo": "14", "nome": "Cross Lander"}, {"codigo": "241", "nome": "D2D Motors"}, {"codigo": "15", "nome": "Daewoo"}, {"codigo": "16", "nome": "Daihatsu"}, {"codigo": "246", "nome": "DFSK"}, {"codigo": "17", "nome": "Dodge"}, {"codigo": "147", "nome": "EFFA"}, {"codigo": "18", "nome": "Engesa"}, {"codigo": "19", "nome": "Envemo"}, {"codigo": "20", "nome": "Ferrari"}, {"codigo": "21", "nome": "Fiat"}, {"codigo": "149", "nome": "Fibravan"}, {"codigo": "22", "nome": "Ford"}, {"codigo": "190", "nome": "FOTON"}, {"codigo": "170", "nome": "Fyber"}, {"codigo": "199", "nome": "GEELY"}, {"codigo": "23", "nome": "GM - Chevrolet"}, {"codigo": "153", "nome": "GREAT WALL"}, {"codigo": "24", "nome": "Gurgel"}, {"codigo": "240", "nome": "GWM"}, {"codigo": "152", "nome": "HAFEI"}, {"codigo": "214", "nome": "HITECH ELECTRIC"}, {"codigo": "25", "nome": "Honda"}, {"codigo": "26", "nome": "Hyundai"}, {"codigo": "27", "nome": "Isuzu"}, {"codigo": "208", "nome": "IVECO"}, {"codigo": "177", "nome": "JAC"}, {"codigo": "28", "nome": "Jaguar"}, {"codigo": "29", "nome": "Jeep"}, {"codigo": "154", "nome": "JINBEI"}, {"codigo": "30", "nome": "JPX"}, {"codigo": "31", "nome": "Kia Motors"}, {"codigo": "32", "nome": "Lada"}, {"codigo": "171", "nome": "LAMBORGHINI"}, {"codigo": "33", "nome": "Land Rover"}, {"codigo": "34", "nome": "Lexus"}, {"codigo": "168", "nome": "LIFAN"}, {"codigo": "127", "nome": "LOBINI"}, {"codigo": "35", "nome": "Lotus"}, {"codigo": "140", "nome": "Mahindra"}, {"codigo": "36", "nome": "Maserati"}, {"codigo": "37", "nome": "Matra"}, {"codigo": "38", "nome": "Mazda"}, {"codigo": "211", "nome": "McLaren"}, {"codigo": "39", "nome": "Mercedes-Benz"}, {"codigo": "40", "nome": "Mercury"}, {"codigo": "167", "nome": "MG"}, {"codigo": "156", "nome": "MINI"}, {"codigo": "41", "nome": "Mitsubishi"}, {"codigo": "42", "nome": "Miura"}, {"codigo": "43", "nome": "Nissan"}, {"codigo": "44", "nome": "Peugeot"}, {"codigo": "45", "nome": "Plymouth"}, {"codigo": "46", "nome": "Pontiac"}, {"codigo": "47", "nome": "Porsche"}, {"codigo": "185", "nome": "RAM"}, {"codigo": "186", "nome": "RELY"}, {"codigo": "48", "nome": "Renault"}, {"codigo": "195", "nome": "Rolls-Royce"}, {"codigo": "49", "nome": "Rover"}, {"codigo": "50", "nome": "Saab"}, {"codigo": "51", "nome": "Saturn"}, {"codigo": "52", "nome": "Seat"}, {"codigo": "247", "nome": "SERES"}, {"codigo": "183", "nome": "SHINERAY"}, {"codigo": "157", "nome": "smart"}, {"codigo": "125", "nome": "SSANGYONG"}, {"codigo": "54", "nome": "Subaru"}, {"codigo": "55", "nome": "Suzuki"}, {"codigo": "165", "nome": "TAC"}, {"codigo": "56", "nome": "Toyota"}, {"codigo": "57", "nome": "Troller"}, {"codigo": "58", "nome": "Volvo"}, {"codigo": "59", "nome": "Vw - Volkswagen"}, {"codigo": "163", "nome": "Wake"}, {"codigo": "120", "nome": "Walk"}]
```

- Modelos da marca VW - VolksWagen (59):
- <https://parallelum.com.br/fipe/api/v1/carros/marcas/59/modelos>

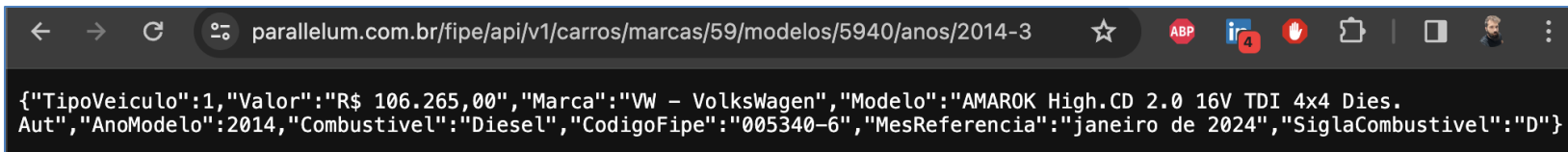
```
← → ↺ 🌐 parallelum.com.br/fipe/api/v1/carros/marcas/59/modelos ☆ 🔍 📄 🗑️ 👤 ⋮

{"modelos":[{"codigo":5585,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x2 Die"}, {"codigo":5586,"nome":"AMAROK CD2.0 16V/S CD2.0 16V TDI 4x4 Die"}, {"codigo":9895,"nome":"AMAROK Comfor. 3.0 V6 TDI 4x4 Dies. Aut."}, {"codigo":8531,"nome":"AMAROK Comfor. CD 2.0 TDI 4x4 Dies. Aut."}, {"codigo":5748,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x2 Diesel"}, {"codigo":5749,"nome":"AMAROK CS2.0 16V/S2.0 16V TDI 4x4 Diesel"}, {"codigo":8532,"nome":"AMAROK Extreme CD 3.0 4x4 TB Dies. Aut."}, {"codigo":7854,"nome":"AMAROK Hig. Extreme CD 2.0 4x4 Dies. Aut."}, {"codigo":7674,"nome":"AMAROK Hig.ULTIMATE CD 2.0 4x4 Dies. Aut."}, {"codigo":5940,"nome":"AMAROK Highline CD 2.0 16V TDI 4x4 Dies. Aut."}, {"codigo":5261,"nome":"AMAROK Highline CD 2.0 16V TDI 4x4 Dies."}, {"codigo":8183,"nome":"AMAROK Highline CD 3.0 4x4 TB Dies. Aut."}, {"codigo":5476,"nome":"AMAROK SE CD 2.0 16V TDI 4x4 Diesel"}, {"codigo":7367,"nome":"AMAROK T. Dark Label CD 2.0 4x4 Dies Aut"}, {"codigo":5350,"nome":"AMAROK Trendline CD 2.0 16V TDI 4x4 Dies"}, {"codigo":6277,"nome":"AMAROK Trendline CD 2.0 TDI 4x4 Dies Aut"}, {"codigo":2359,"nome":"Apolo GL 1.8"}, {"codigo":2360,"nome":"Apolo GLS/ Vip 1.8"}, {"codigo":2362,"nome":"Bora 2.0 8v Comfortline Aut."}, {"codigo":2363,"nome":"Bora 2.0 8v Comfortline Mec."}, {"codigo":2361,"nome":"Bora 2.0/ 2.0 Flex 8v Aut."}, {"codigo":2364,"nome":"Bora 2.0/ 2.0 Flex 8v Mec."}, {"codigo":2365,"nome":"Caravelle 2.4 Diesel"}, {"codigo":2366,"nome":"Corrado 2.0 Turbo"}, {"codigo":2367,"nome":"Corrado G-60 2.8"}, {"codigo":7077,"nome":"CROSSFOX 1.6 T. Flex 16V 5p"}, {"codigo":7078,"nome":"CROSSFOX I MOTION 1.6 T. Flex 16V 5p"}, {"codigo":2368,"nome":"CROSSFOX 1.6 Mi Total Flex 8V 5p"}, {"codigo":5983,"nome":"CROSSFOX I MOTION 1.6 Mi T. Flex 8V 5p"}, {"codigo":10368,"nome":"Delivery Express Turbo Diesel (E6)"}, {"codigo":10137,"nome":"Delivery Express- Turbo Diesel"}, {"codigo":4857,"nome":"EOS Cab. 2.0 Turbo FSI Tiptronic"}, {"codigo":2369,"nome":"Eurovan 2.4 Diesel"}, {"codigo":5082,"nome":"Fox 1.0 Mi Total Flex 8V 3p"}, {"codigo":5083,"nome":"Fox 1.0 Mi Total Flex 8V 5p"}, {"codigo":5084,"nome":"Fox 1.6 Mi I MOTION Total Flex 8V 5p"}, {"codigo":5085,"nome":"Fox 1.6 Mi Total Flex 8V 5p"}, {"codigo":6540,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 3p"}, {"codigo":6540,"nome":"Fox BLUEMOTION 1.0 Mi Total Flex 12V 5p"}, {"codigo":6639,"nome":"Fox BlueMotion 1.6 Mi Total Flex 3p"}, {"codigo":6640,"nome":"Fox BlueMotion 1.6 Mi Total Flex 5p"}, {"codigo":2370,"nome":"Fox City 1.0 Mi/ 1.0Mi Total Flex 8V 5p"}, {"codigo":2371,"nome":"Fox City 1.0Mi/ 1.0Mi Total Flex 8V 3p"}, {"codigo":7269,"nome":"Fox Comfortline 1.0 Flex 12V 5p"}, {"codigo":6934,"nome":"Fox Comfortline 1.0 Flex 8V 5p"}, {"codigo":6935,"nome":"Fox Comfortline 1.6 Flex 8V 5p"}, {"codigo":6936,"nome":"Fox Comfortline I Motion 1.6 Flex 8V 5p"}, {"codigo":8112,"nome":"Fox Connect 1.6 Flex 8V 5p"}, {"codigo":8113,"nome":"Fox Connect I Motion 1.6 Flex 8V 5p"}, {"codigo":4668,"nome":"Fox extreme 1.6 Mi Total Flex 8V 5p"}, {"codigo":6937,"nome":"Fox Highline I MOTION 1.6 Flex 16V 5p"}, {"codigo":6938,"nome":"Fox Highline 1.6 Flex 16V 5p"}, {"codigo":7172,"nome":"Fox PEPPER 1.6 Flex 16V 5p"}, {"codigo":7173,"nome":"Fox PEPPER I MOTION 1.6 Flex 16V 5p"}, {"codigo":2372,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 3p"}, {"codigo":2373,"nome":"Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 4p"}, {"codigo":2374,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 3p"}, {"codigo":2375,"nome":"Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 4p"}, {"codigo":5087,"nome":"Fox PRIME/Hghi. I MOTION 1.6 T.Flex 8V 5p"}, {"codigo":5086,"nome":"Fox PRIME/Higl. 1.6 Total Flex 8V 5p"}, {"codigo":5599,"nome":"Fox Rock in Rio 1.6 Mi Total Flex 8V 5p"}, {"codigo":4349,"nome":"Fox Route 1.0 Mi Total Flex 8V 3p"}, {"codigo":4350,"nome":"Fox Route 1.0 Mi Total Flex 8V 5p"}, {"codigo":4351,"nome":"Fox Route 1.6 Mi Total Flex 8V 3p"}, {"codigo":4352,"nome":"Fox Route 1.6 Mi Total Flex 8V 5p"}, {"codigo":7637,"nome":"Fox RUN 1.6 Flex 8V 5p"}, {"codigo":6667,"nome":"Fox SELEÇÃO 1.0 Total Flex 8V 5p"}, {"codigo":6668,"nome":"Fox SELEÇÃO 1.6 Total Flex 8V 5p"}, {"codigo":6784,"nome":"Fox SELEÇÃO I MOTION 1.6 Mi T. Flex 8V 5p"}, {"codigo":2379,"nome":"Fox Sportline/Sports 1.0 Tot.Flex 8V 4p"}, {"codigo":2376,"nome":"Fox Sportline/Sports 1.0/1.0 Tot.Flex 3p"}, {"codigo":2377,"nome":"Fox Sportline/Sports 1.6/1.6 Tot.Flex 3p"}, {"codigo":2378,"nome":"Fox Sportline/Sports 1.6/1.6 Tot.Flex 4p"}, {"codigo":4928,"nome":"Fox SUNRISE 1.0 Mi Total Flex 8V 5p"}, {"codigo":7410,"nome":"Fox TRACK 1.0 Flex 12V 5p"}, {"codigo":7295,"nome":"Fox Trendline 1.0 Flex 12V 5p"}, {"codigo":6939,"nome":"Fox Trendline 1.0 Flex 8V 5p"}, {"codigo":6940,"nome":"Fox Trendline 1.6 Flex 8V 5p"}, {"codigo":8113,"nome":"Fox Xtreme 1.6 Flex 8V 12p"}, {"codigo":2380,"nome":"Fusca"}, {"codigo":7079,"nome":"Fusca 2.0 R-Line TSI 16V Aut."}, {"codigo":6278,"nome":"Fusca 2.0 TSI 16V Aut."}, {"codigo":6279,"nome":"Fusca 2.0 TSI 16V Mec."}, {"codigo":6280,"nome":"Gol (novo) 1.0 Mi Total Flex 8V 2p"}, {"codigo":4689,"nome":"Gol (novo) 1.0 Mi Total Flex 8V 4p"}, {"codigo":6281,"nome":"Gol (novo) 1.6 Mi Total Flex 8V 2p"}, {"codigo":4691,"nome":"Gol (novo) 1.6 Mi Total Flex 8V 4p"}, {"codigo":4698,"nome":"Gol (novo) 1.6 Power/High T.Flex 8V 4p"}, {"codigo":6323,"nome":"Gol 1.0 Flex 12V 5p"}, {"codigo":2381,"nome":"Gol 1.0 FLM/ Highway/ Sport 16V 2/4p"}, {"codigo":2382,"nome":"Gol 1.0 Plus 16V 2p"}, {"codigo":2383,"nome":"Gol 1.0 Plus 16V 4p"}, {"codigo":2384,"nome":"Gol 1.0 Plus 8V 2p"}, {"codigo":2385,"nome":"Gol 1.0 Plus 8V 4p"}, {"codigo":2386,"nome":"Gol 1.0 Power 16V 76cv 4p"}, {"codigo":5941,"nome":"Gol 1.0 Total Flex 8V 5p (25 Anos)"}, {"codigo":2387,"nome":"Gol 1.0 Trend/ Power 8V 2p"}, {"codigo":2388,"nome":"Gol 1.0 Trend/ Power 8V 4p"}, {"codigo":5114,"nome":"Gol 1.6 I MOTI. Power/High T. Flex 8V 4p"}, {"codigo":6454,"nome":"Gol 1.6 Mi I MOTION Total Flex 8V 3p"}, {"codigo":2389,"nome":"Gol 1.6 Mi Total Flex 8V 2p"}, {"codigo":2390,"nome":"Gol 1.6 Mi Plus Total Flex 8V 4p"}, {"codigo":2391,"nome":"Gol 1.6 Mi Power Total Flex 8V 4p"}, {"codigo":5484,"nome":"Gol 1.6 Mi Rallye I MOTION T. Flex 8V 4p"}, {"codigo":2392,"nome":"Gol 1.6 Mi Rallye Total Flex 8V 4p"}, {"codigo":2393,"nome":"Gol 1.6 Mi/ 1.6i 2p"}, {"codigo":2394,"nome":"Gol 1.6 Mi/ Power 1.6 Mi 4p"}, {"codigo":8463,"nome":"Gol 1.6 MSI Flex 16V Aut."}, {"codigo":8324,"nome":"Gol 1.6 MSI Flex 8V 5p"}, {"codigo":2395,"nome":"Gol 1.8 Mi"}, {"codigo":2396,"nome":"Gol 1.8 Mi 4p"}, {"codigo":2397,"nome":"Gol 1.8 Mi Power Total Flex 8V 4p"}, {"codigo":2398,"nome":"Gol 1.8 Mi Rallye Total Flex 8V 4p"}, {"codigo":2399,"nome":"Gol 1000 (modelo antigo)"}, {"codigo":2400,"nome":"Gol 1000 Mi 16V 2p Turbo"}, {"codigo":2401,"nome":"Gol 1000 Mi 16V 4p Turbo"}, {"codigo":2402,"nome":"Gol 1000 Mi 16V/ Ouro 2p"}, {"codigo":2403,"nome":"Gol 1000 Mi 16V/ Ouro 4p"}, {"codigo":2404,"nome":"Gol 1000 Mi 2p / 1000i"}, {"codigo":2405,"nome":"Gol 1000 Mi 4p"}, {"codigo":2406,"nome":"Gol 1000 Mi Plus 16V 2p e 4p"}, {"codigo":2407,"nome":"Gol 1000 Mi Plus 8V 2p e 4p"}, {"codigo":2408,"nome":"Gol 1000i Plus 2p"}, {"codigo":2409,"nome":"Gol 2.0 Mi 2p"}, {"codigo":2410,"nome":"Gol 2.0 Mi 4p"}, {"codigo":5750,"nome":"Gol BLACK 1.0 Mi Total Flex 8V 4p"}, {"codigo":2413,"nome":"Gol City (Trend) 1.0 Mi Total Flex 8V 2p"}, {"codigo":3832,"nome":"Gol City (Trend) 1.6 Mi T. Flex 8V 2p"}, {"codigo":2417,"nome":"Gol City (Trend) 1.6 Mi T. Flex 8V 4p"}, {"codigo":2414,"nome":"Gol City (Trend)/Titan 1.0 T. Flex 8V 4p"}, {"codigo":2411,"nome":"Gol City 1.0 Mi 8V 2p"}, {"codigo":2412,"nome":"Gol City 1.0 Mi 8V 4p"}, {"codigo":9098,"nome":"Gol City 1.0 Total Flex 12V 2p"}, {"codigo":8840,"nome":"Gol City 1.0 Total Flex 12V 4p"}, {"codigo":2415,"nome":"Gol City 1.6 Mi 8V 2p"}, {"codigo":2416,"nome":"Gol City 1.6 Mi 8V 4p"}, {"codigo":2418,"nome":"Gol CL 1.6 Mi 2p e 4p"}, {"codigo":2419,"nome":"Gol CL 1.8 Mi 2p e 4p"}, {"codigo":2420,"nome":"Gol CLi / CL 1.8"}, {"codigo":2421,"nome":"Gol CLi / CL/ Copa/ Stones 1.6"}, {"codigo":6791,"nome":"Gol Comfort. I MOTION 1.6 T. Flex 8V 5p"}, {"codigo":6792,"nome":"Gol Comfort. I MOTION 1.6 T. Flex 8V 5p"}, {"codigo":7521,"nome":"Gol Comfortline 1.0 T. Flex 12V 5p"}, {"codigo":6787,"nome":"Gol Comfortline 1.0 T. Flex 8V 3p"}, {"codigo":6788,"nome":"Gol Comfortline 1.0 T. Flex 8V 5p"}, {"codigo":6789,"nome":"Gol Comfortline 1.6 T. Flex 8V 3p"}, {"codigo":6790,"nome":"Gol Comfortline 1.6 T. Flex 8V 5p"}, {"codigo":3058,"nome":"Gol CRAI 1.0 Mi Total Flex 8V 5p"}]}
```

Exercício

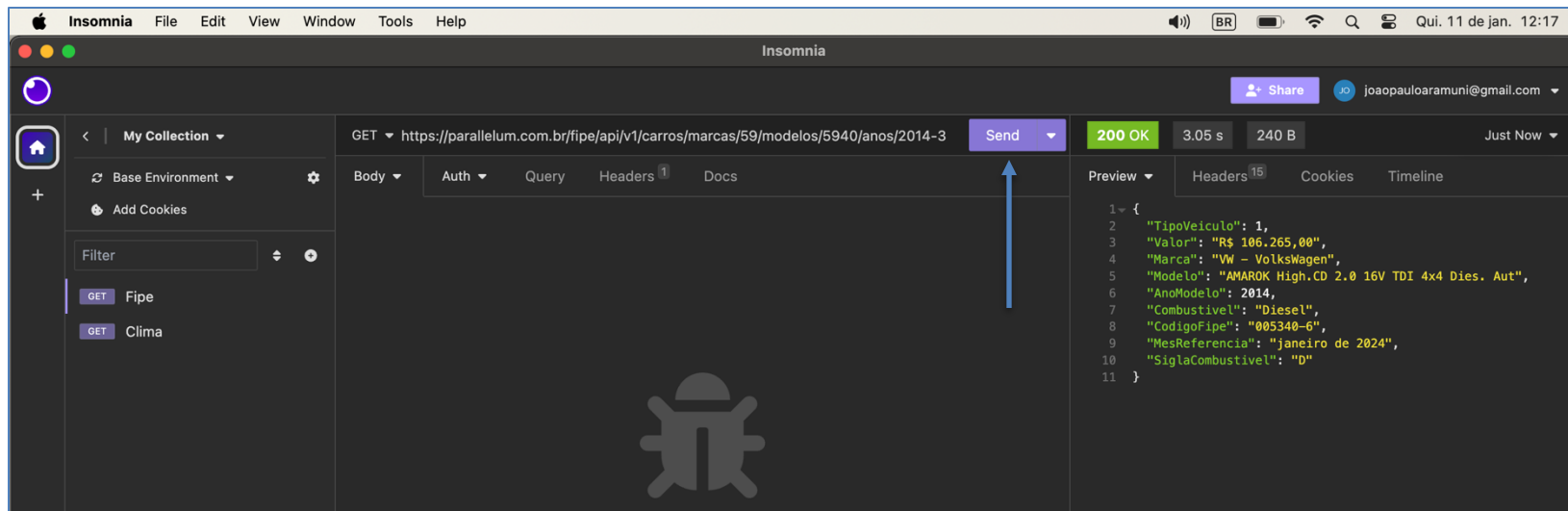
- **FIPE API HTTP REST**

- Por fim, o preço corrente da Tabela FIPE para um veículo/modelo/ano.
- Valor de uma AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut ano 2014 a Diesel:
 - <https://parallelum.com.br/fipe/api/v1/carros/marcas/59/modelos/5940/anos/2014-3>
 - **R\$ 106.265,00**



Exercício

- Faça o teste da URL também em uma plataforma de APIs, como Postman e Insomnia:

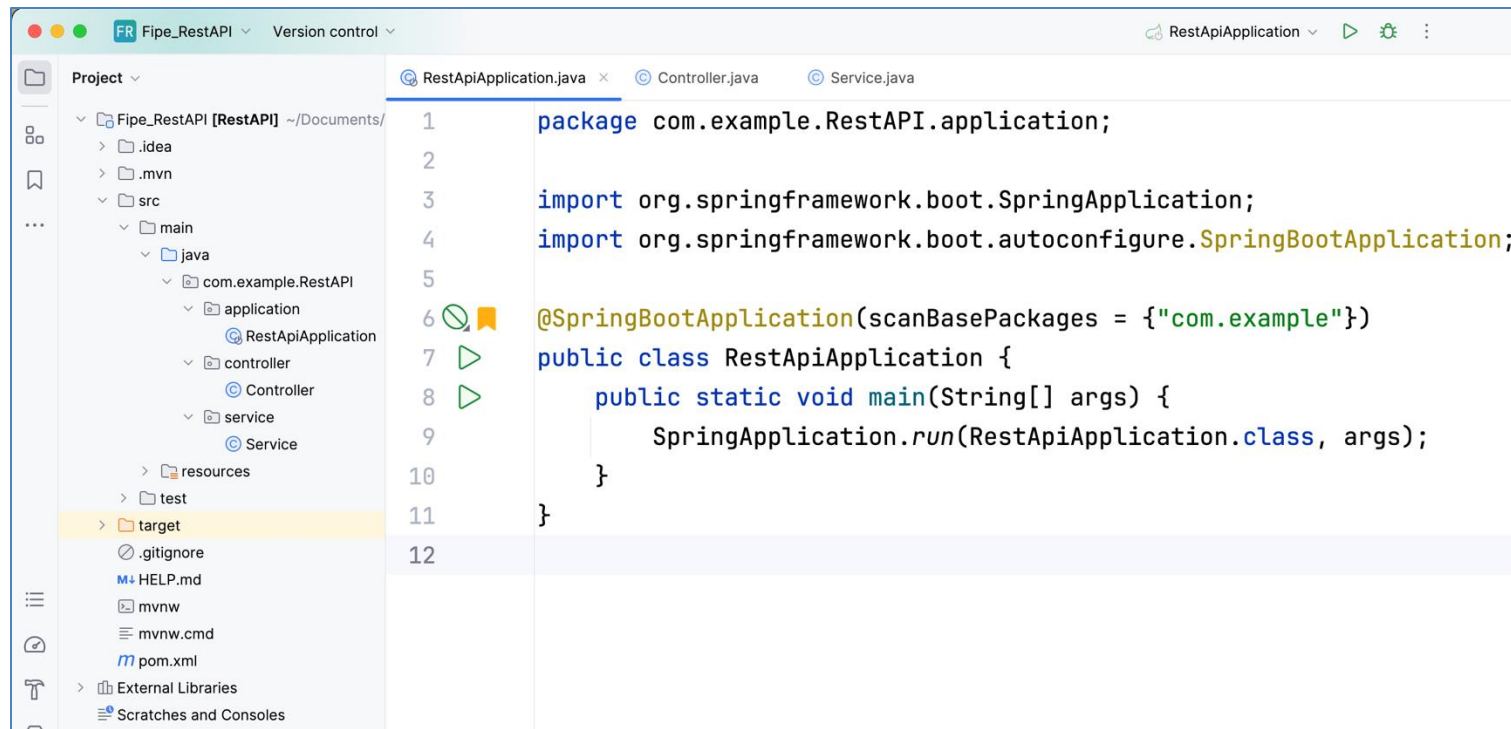


Exercício

- Uma vez que a chamada à API está funcionando no browser, podemos levar os links/URLs para o nosso código Java.
- Faremos uma estrutura de projeto muito semelhante ao exemplo anterior, com os pacotes **application**, **controller** e **service**.

Exercício

- Classe **RestApiApplication** com o método **main**:



Classe **Controller** com os **endpoints** e as chamadas para a classe **Service**:

RestApiApplication.java Controller.java x Service.java

```
1 package com.example.RestAPI.controller;
2 import com.example.RestAPI.service.Service;
3 import org.springframework.web.bind.annotation.PathVariable;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 @RestController
8 public class Controller {
9     Service service = new Service();
10
11     @GetMapping("/marcas")
12     public String consultarMarcas(){
13         return service.consultarMarcas();
14     }
15     @GetMapping("/modelos/{marca}")
16     public String consultarModelos(@PathVariable int marca){
17         return service.consultarModelos(marca);
18     }
19     @GetMapping("/anos/{marca}/{modelo}")
20     public String consultarAnos(@PathVariable int marca, @PathVariable int modelo){
21         return service.consultarAnos(marca, modelo);
22     }
23     @GetMapping("/valor/{marca}/{modelo}/{ano}")
24     public String consultarValor(@PathVariable int marca, @PathVariable int modelo, @PathVariable String ano){
25         return service.consultarValor(marca, modelo, ano);
26     }
27 }
```

Repare que agora estamos passando parâmetros na URL por meio da annotation **@PathVariable**

Temos 4 endpoints e consequentemente 4 rotas

Classe **Service** contendo os serviços de consulta de marcas, modelos, anos e valor:

```
RestApiApplication.java  Controller.java  Service.java x

1  package com.example.RestAPI.service;
2  import org.springframework.http.ResponseEntity;
3  import org.springframework.web.client.RestTemplate;
4
5  public class Service {
6
7      private String consultarURL(String apiUrl){
8          String dados = "";
9          RestTemplate restTemplate = new RestTemplate();
10         ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
11         if (responseEntity.getStatusCode().is2xxSuccessful()) {
12             dados = responseEntity.getBody();
13         } else {
14             dados = "Falha ao obter dados. Código de status: " + responseEntity.getStatusCode();
15         }
16         return dados;
17     }
18     public String consultarMarcas() {
19         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas");
20     }
21     public String consultarModelos(int id) {
22         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+id+"/modelos");
23     }
24     public String consultarAnos(int marca, int modelo) {
25         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos");
26     }
27     public String consultarValor(int marca, int modelo, String ano) {
28         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos/"+ano);
29     }
30 }
```

Extraindo a lógica
para não duplicar
código em todos
os métodos

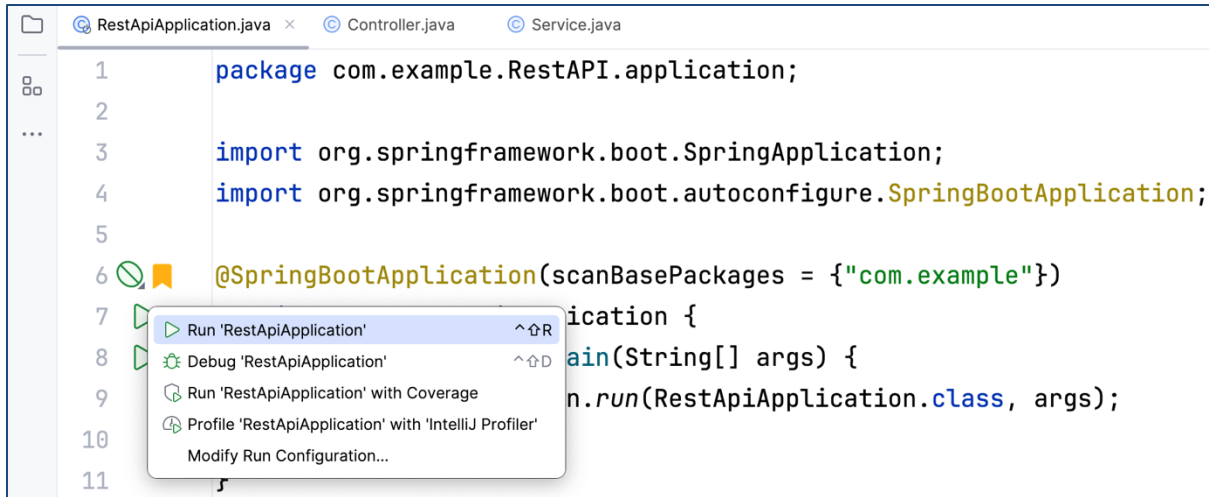
Classe **Service** contendo os serviços de consulta de marcas, modelos, anos e valor:

```
RestApiApplication.java  Controller.java  Service.java x
1  package com.example.RestAPI.service;
2  import org.springframework.http.ResponseEntity;
3  import org.springframework.web.client.RestTemplate;
4
5  public class Service {
6
7      private String consultarURL(String apiUrl){
8          String dados = "";
9          RestTemplate restTemplate = new RestTemplate();
10         ResponseEntity<String> responseEntity = restTemplate.getForEntity(apiUrl, String.class);
11         if (responseEntity.getStatusCode().is2xxSuccessful()) {
12             dados = responseEntity.getBody();
13         } else {
14             dados = "Falha ao obter dados. Código de status: " + responseEntity.getStatusCode();
15         }
16         return dados;
17     }
18     public String consultarMarcas() {
19         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas");
20     }
21     public String consultarModelos(int id) {
22         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+id+"/modelos");
23     }
24     public String consultarAnos(int marca, int modelo) {
25         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos");
26     }
27     public String consultarValor(int marca, int modelo, String ano) {
28         return consultarURL( apiUrl: "https://parallelum.com.br/fipe/api/v1/carros/marcas/"+marca+"/modelos/"+modelo+"/anos/"+ano);
29     }
30 }
```

Serviços/regras de
negócio separados
do controller

Exercício

- Por fim, basta rodar a aplicação e ver o resultado, em formato **JSON**, no browser ou no Postman/Insomnia:



Marcas

← → ↺ ⓘ localhost:8080/marcas ☆ ABB ⓘ ⓘ ⓘ ⓘ ⓘ ⓘ ⓘ

```
[{"codigo": "1", "nome": "Acura"}, {"codigo": "2", "nome": "Agrale"}, {"codigo": "3", "nome": "Alfa Romeo"}, {"codigo": "4", "nome": "AM Gen"}, {"codigo": "5", "nome": "Asia Motors"}, {"codigo": "189", "nome": "ASTON MARTIN"}, {"codigo": "6", "nome": "Audi"}, {"codigo": "207", "nome": "Baby"}, {"codigo": "7", "nome": "BMW"}, {"codigo": "8", "nome": "BRM"}, {"codigo": "123", "nome": "Bugre"}, {"codigo": "238", "nome": "BYD"}, {"codigo": "236", "nome": "CAB Motors"}, {"codigo": "10", "nome": "Cadillac"}, {"codigo": "245", "nome": "Caoa Chery"}, {"codigo": "161", "nome": "Caoa Chery/Chery"}, {"codigo": "11", "nome": "CBT Jipe"}, {"codigo": "136", "nome": "CHANA"}, {"codigo": "182", "nome": "CHANGAN"}, {"codigo": "12", "nome": "Chrysler"}, {"codigo": "13", "nome": "Citroën"}, {"codigo": "14", "nome": "Cross Lander"}, {"codigo": "241", "nome": "D2D Motors"}, {"codigo": "15", "nome": "Daewoo"}, {"codigo": "16", "nome": "Daihatsu"}, {"codigo": "246", "nome": "DFSK"}, {"codigo": "17", "nome": "Dodge"}, {"codigo": "147", "nome": "EFA"}, {"codigo": "18", "nome": "Engesa"}, {"codigo": "19", "nome": "Envemo"}, {"codigo": "20", "nome": "Ferrari"}, {"codigo": "21", "nome": "Fiat"}, {"codigo": "149", "nome": "Fibravan"}, {"codigo": "22", "nome": "Ford"}, {"codigo": "190", "nome": "FOTON"}, {"codigo": "170", "nome": "Fyber"}, {"codigo": "199", "nome": "GEELY"}, {"codigo": "23", "nome": "GM - Chevrolet"}, {"codigo": "153", "nome": "GREAT WALL"}, {"codigo": "24", "nome": "Gurgel"}, {"codigo": "240", "nome": "GWM"}, {"codigo": "152", "nome": "HAFEI"}, {"codigo": "214", "nome": "HITECH ELECTRIC"}, {"codigo": "25", "nome": "Honda"}, {"codigo": "26", "nome": "Hyundai"}, {"codigo": "27", "nome": "Isuzu"}, {"codigo": "208", "nome": "IVECO"}, {"codigo": "177", "nome": "JAC"}, {"codigo": "28", "nome": "Jaguar"}, {"codigo": "29", "nome": "Jeep"}, {"codigo": "154", "nome": "JINBEI"}, {"codigo": "30", "nome": "JPX"}, {"codigo": "31", "nome": "Kia Motors"}, {"codigo": "32", "nome": "Lada"}, {"codigo": "171", "nome": "LAMBORGHINI"}, {"codigo": "33", "nome": "Land Rover"}, {"codigo": "34", "nome": "Lexus"}, {"codigo": "168", "nome": "LIFAN"}, {"codigo": "127", "nome": "LOBINI"}, {"codigo": "35", "nome": "Lotus"}, {"codigo": "140", "nome": "Mahindra"}, {"codigo": "36", "nome": "Maserati"}, {"codigo": "37", "nome": "Matra"}, {"codigo": "38", "nome": "Mazda"}, {"codigo": "211", "nome": "McLaren"}, {"codigo": "39", "nome": "Mercedes-Benz"}, {"codigo": "40", "nome": "Mercury"}, {"codigo": "167", "nome": "MG"}, {"codigo": "156", "nome": "MINI"}, {"codigo": "41", "nome": "Mitsubishi"}, {"codigo": "42", "nome": "Miura"}, {"codigo": "43", "nome": "Nissan"}, {"codigo": "44", "nome": "Peugeot"}, {"codigo": "45", "nome": "Plymouth"}, {"codigo": "46", "nome": "Pontiac"}, {"codigo": "47", "nome": "Porsche"}, {"codigo": "185", "nome": "RAM"}, {"codigo": "186", "nome": "RELY"}, {"codigo": "48", "nome": "Renault"}, {"codigo": "195", "nome": "Rolls-Royce"}, {"codigo": "49", "nome": "Rover"}, {"codigo": "50", "nome": "Saab"}, {"codigo": "51", "nome": "Saturn"}, {"codigo": "52", "nome": "Seat"}, {"codigo": "247", "nome": "SERES"}, {"codigo": "183", "nome": "SHINERAY"}, {"codigo": "157", "nome": "smart"}, {"codigo": "125", "nome": "SSANGYONG"}, {"codigo": "54", "nome": "Subaru"}, {"codigo": "55", "nome": "Suzuki"}, {"codigo": "165", "nome": "TAC"}, {"codigo": "56", "nome": "Toyota"}, {"codigo": "57", "nome": "Troller"}, {"codigo": "58", "nome": "Volvo"}, {"codigo": "59", "nome": "VW - VolksWagen"}, {"codigo": "163", "nome": "Wake"}, {"codigo": "120", "nome": "Walk"}]
```

GET http://localhost:8080/marcas Send 200 OK 1.35 s 3.1 KB

Body Auth Query Headers 1 Do Preview 3 Headers 3 Cookies



Enter a URL and send to get a response

Select a body type from above to send data in the body of a request

Introduction to Insomnia ↗

1 {
2 {
3 "codigo": "1",
4 "nome": "Acura"
5 },
6 {
7 "codigo": "2",
8 "nome": "Agrale"
9 },
10 {
11 "codigo": "3",
12 "nome": "Alfa Romeo"
13 },
14 {
15 "codigo": "4",
16 "nome": "AM Gen"
17 },
18 {
19 "codigo": "5",
20 "nome": "Asia Motors"
21 },
22 {
23 "codigo": "189",
24 "nome": "ASTON MARTIN"
25 },
26 {
27 "codigo": "6",
28 "nome": "Audi"
29 },
30 {
31 "codigo": "207",
32 "nome": "Baby"
33 },
34 {
35 "codigo": "7",
36 "nome": "BMW"
37 },

Modelos

← → ↺

localhost:8080/modelos/59

☆

ABP

🔗

🔥

📄

🖨

👤

⋮

```
{
  "modelos": [
    {
      "codigo": "5585",
      "nome": "AMAROK CD2.0 16V/S CD2.0 16V TDI 4x2 Die"
    },
    {
      "codigo": "5586",
      "nome": "AMAROK CD2.0 16V/S CD2.0 16V TDI 4x4 Die"
    },
    {
      "codigo": "9895",
      "nome": "AMAROK Comfor. 3.0 V6 TDI 4x4 Dies. Aut."
    },
    {
      "codigo": "8531",
      "nome": "AMAROK Comfor. CD 2.0 TDI 4x4 Dies. Aut."
    },
    {
      "codigo": "5748",
      "nome": "AMAROK CS2.0 16V/S2.0 16V TDI 4x2 Diesel"
    },
    {
      "codigo": "5749",
      "nome": "AMAROK CS2.0 16V/S2.0 16V TDI 4x4 Diesel"
    },
    {
      "codigo": "8532",
      "nome": "AMAROK Extreme CD 3.0 4x4 TB Dies. Aut."
    },
    {
      "codigo": "7854",
      "nome": "AMAROK Hig. Extreme CD 2.0 4x4 Dies. Aut"
    },
    {
      "codigo": "7674",
      "nome": "AMAROK Hig.ULTIMATE CD 2.0 4x4 Dies. Aut"
    },
    {
      "codigo": "5940",
      "nome": "AMAROK High.CD 2.0 16V TDI 4x4 Dies. Aut"
    },
    {
      "codigo": "5261",
      "nome": "AMAROK Highline CD 2.0 16V TDI 4x4 Dies."
    },
    {
      "codigo": "8183",
      "nome": "AMAROK Highline CD 3.0 4x4 TB Dies. Aut."
    },
    {
      "codigo": "5476",
      "nome": "AMAROK SE CD 2.0 16V TDI 4x4 Diesel"
    },
    {
      "codigo": "7367",
      "nome": "AMAROK T. Dark Label CD 2.0 4x4 Dies Aut"
    },
    {
      "codigo": "5350",
      "nome": "AMAROK Trendline CD 2.0 16V TDI 4x4 Dies"
    },
    {
      "codigo": "6277",
      "nome": "AMAROK Trendline CD 2.0 TDI 4X4 Dies Aut"
    },
    {
      "codigo": "2359",
      "nome": "Apolo GL 1.8"
    },
    {
      "codigo": "2360",
      "nome": "Apolo GLS/ Vip 1.8"
    },
    {
      "codigo": "2362",
      "nome": "Bora 2.0 8v Comfortline Aut."
    },
    {
      "codigo": "2363",
      "nome": "Bora 2.0 8v Comfortline Mec."
    },
    {
      "codigo": "2361",
      "nome": "Bora 2.0/ 2.0 Flex 8v Aut."
    },
    {
      "codigo": "2364",
      "nome": "Bora 2.0/ 2.0 Flex 8v Mec."
    },
    {
      "codigo": "2365",
      "nome": "Caravelle 2.4 Diesel"
    },
    {
      "codigo": "2366",
      "nome": "Corrado 2.0 Turbo"
    },
    {
      "codigo": "2367",
      "nome": "Corrado G-60 2.8"
    },
    {
      "codigo": "7077",
      "nome": "CROSSFOX 1.6 T. Flex 16V 5p"
    },
    {
      "codigo": "7078",
      "nome": "CROSSFOX I MOTION 1.6 T. Flex 16V 5p"
    },
    {
      "codigo": "2368",
      "nome": "CROSSFOX 1.6 Mi Total Flex 8V 5p"
    },
    {
      "codigo": "5983",
      "nome": "CROSSFOX I MOTION 1.6 Mi T. Flex 8V 5p"
    },
    {
      "codigo": "10368",
      "nome": "Delivery Express Turbo Diesel (E6)"
    },
    {
      "codigo": "10137",
      "nome": "Delivery Express+ Turbo Diesel"
    },
    {
      "codigo": "4857",
      "nome": "EOS Cab. 2.0 Turbo FSI Tiptronic"
    },
    {
      "codigo": "2369",
      "nome": "Eurovan 2.4 Diesel"
    },
    {
      "codigo": "5082",
      "nome": "Fox 1.0 Mi Total Flex 8V 3p"
    },
    {
      "codigo": "5083",
      "nome": "Fox 1.0 Mi Total Flex 8V 5p"
    },
    {
      "codigo": "5084",
      "nome": "Fox 1.6 Mi I MOTION Total Flex 8V 5p"
    },
    {
      "codigo": "5085",
      "nome": "Fox 1.6 Mi Total Flex 8V 5p"
    },
    {
      "codigo": "6548",
      "nome": "Fox BLUEMOTION 1.0 Mi Total Flex 12V 3p"
    },
    {
      "codigo": "6549",
      "nome": "Fox BLUEMOTION 1.0 Mi Total Flex 12V 5p"
    },
    {
      "codigo": "6039",
      "nome": "Fox BlueMotion 1.6 Mi Total Flex 3p"
    },
    {
      "codigo": "6040",
      "nome": "Fox BlueMotion 1.6 Mi Total Flex 5p"
    },
    {
      "codigo": "2370",
      "nome": "Fox City 1.0 Mi/ 1.0Mi Total Flex 8V 5p"
    },
    {
      "codigo": "2371",
      "nome": "Fox City 1.0Mi/ 1.0Mi Total Flex 8V 3p"
    },
    {
      "codigo": "7269",
      "nome": "Fox Comfortline 1.0 Flex 12V 5p"
    },
    {
      "codigo": "6934",
      "nome": "Fox Comfortline 1.0 Flex 8V 5p"
    },
    {
      "codigo": "6935",
      "nome": "Fox Comfortline 1.6 Flex 8V 5p"
    },
    {
      "codigo": "6936",
      "nome": "Fox Comfortline I Motion 1.6 Flex 8V 5p"
    },
    {
      "codigo": "8112",
      "nome": "Fox Connect 1.6 Flex 8V 5p"
    },
    {
      "codigo": "8133",
      "nome": "Fox Connect I Motion 1.6 Flex 8V 5p"
    },
    {
      "codigo": "4668",
      "nome": "Fox extreme 1.6 Mi Total Flex 8V 5p"
    },
    {
      "codigo": "6937",
      "nome": "Fox Highline I MOTION 1.6 Flex 16V 5p"
    },
    {
      "codigo": "6938",
      "nome": "Fox Highline 1.6 Flex 16V 5p"
    },
    {
      "codigo": "7172",
      "nome": "Fox PEPPER 1.6 Flex 16V 5p"
    },
    {
      "codigo": "7173",
      "nome": "Fox PEPPER I MOTION 1.6 Flex 16V 5p"
    },
    {
      "codigo": "2372",
      "nome": "Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 3p"
    },
    {
      "codigo": "2373",
      "nome": "Fox Plus 1.0Mi/ 1.0Mi Total Flex 8V 4p"
    },
    {
      "codigo": "2374",
      "nome": "Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 3p"
    },
    {
      "codigo": "2375",
      "nome": "Fox Plus 1.6Mi/ 1.6Mi Total Flex 8V 4p"
    },
    {
      "codigo": "5087",
      "nome": "Fox PRIME/Hghi. IMOTION 1.6 T.Flex 8V 5p"
    }
  ]
}
```

GET

http://localhost:8080/modelos/59

Send

200 OK

1.46 s

33.2 KB

Body

Auth

Query

Headers1

Docs

Preview

Headers3

Cookies

Timeline

Enter a URL and send to get a response

Select a body type from above to send data in the body of a request

Introduction to Insomnia

```
1 {
2   "modelos": [
3     {
4       "codigo": 5585,
5       "nome": "AMAROK CD2.0 16V/S CD2.0 16V TDI 4x2 Die"
6     },
7     {
8       "codigo": 5586,
9       "nome": "AMAROK CD2.0 16V/S CD2.0 16V TDI 4x4 Die"
10    },
11    {
12      "codigo": 9895,
13      "nome": "AMAROK Comfor. 3.0 V6 TDI 4x4 Dies. Aut."
14    },
15    {
16      "codigo": 8531,
17      "nome": "AMAROK Comfor. CD 2.0 TDI 4x4 Dies. Aut."
18    },
19    {
20      "codigo": 5748,
21      "nome": "AMAROK CS2.0 16V/S2.0 16V TDI 4x2 Diesel"
22    },
23    {
24      "codigo": 5749,
25      "nome": "AMAROK CS2.0 16V/S2.0 16V TDI 4x4 Diesel"
26    },
27    {
28      "codigo": 8532,
29      "nome": "AMAROK Extreme CD 3.0 4x4 TB Dies. Aut."
30    },
31    {
32      "codigo": 7854,
33      "nome": "AMAROK Hig. Extreme CD 2.0 4x4 Dies. Aut"
34    },
35    {
36      "codigo": 7674,
37      "nome": "AMAROK Hig.ULTIMATE CD 2.0 4x4 Dies. Aut"
38    },
39    {
```

Anos

←

→

↺

localhost:8080/anos/59/5940

☆

ABP

in

7

🔥

📁

📄

👤

[{"codigo":"2022-3","nome":"2022 Diesel"}, {"codigo":"2021-3","nome":"2021 Diesel"}, {"codigo":"2020-3","nome":"2020 Diesel"}, {"codigo":"2019-3","nome":"2019 Diesel"}, {"codigo":"2018-3","nome":"2018 Diesel"}, {"codigo":"2017-3","nome":"2017 Diesel"}, {"codigo":"2016-3","nome":"2016 Diesel"}, {"codigo":"2015-3","nome":"2015 Diesel"}, {"codigo":"2014-3","nome":"2014 Diesel"}, {"codigo":"2013-3","nome":"2013 Diesel"}, {"codigo":"2012-3","nome":"2012 Diesel"}, {"codigo":"2011-3","nome":"2011 Diesel"}]

GET http://localhost:8080/anos/59/5940 Send 200 OK 549 ms 493 B

Body Auth Query Headers 1 Docs



Enter a URL and send to get a response

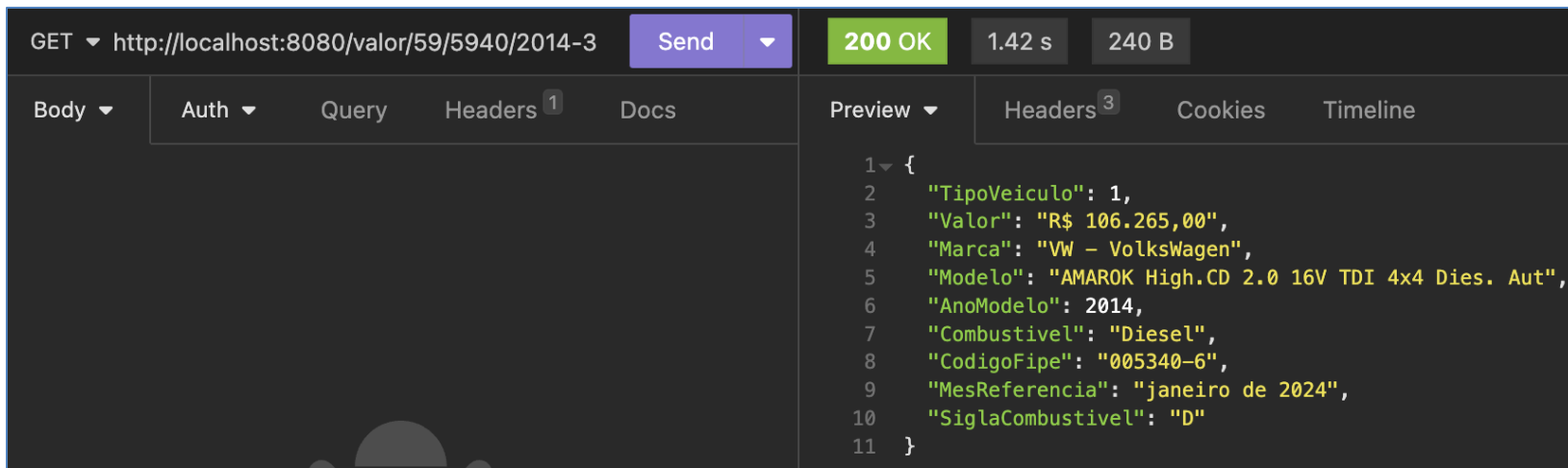
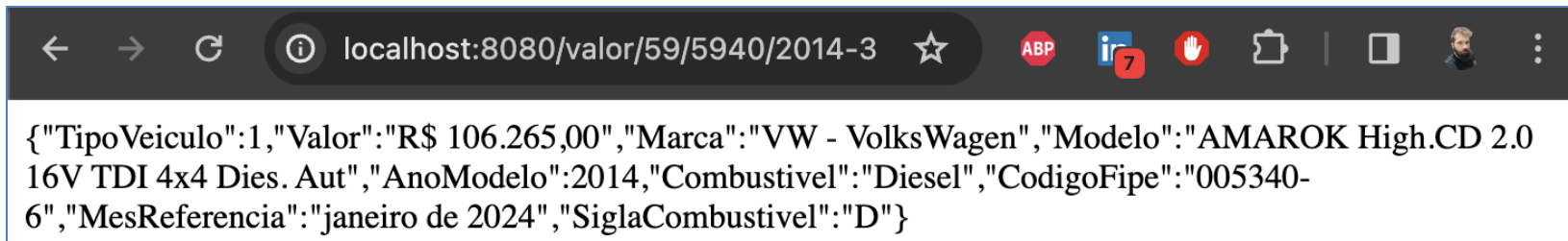
Select a body type from above to send data in the body of a request

Introduction to Insomnia ↗

Preview Headers 3 Cookies Timeline

1 {
2 {
3 "codigo": "2022-3",
4 "nome": "2022 Diesel"
5 },
6 {
7 "codigo": "2021-3",
8 "nome": "2021 Diesel"
9 },
10 {
11 "codigo": "2020-3",
12 "nome": "2020 Diesel"
13 },
14 {
15 "codigo": "2019-3",
16 "nome": "2019 Diesel"
17 },
18 {
19 "codigo": "2018-3",
20 "nome": "2018 Diesel"
21 },
22 {
23 "codigo": "2017-3",
24 "nome": "2017 Diesel"
25 },
26 {
27 "codigo": "2016-3",
28 "nome": "2016 Diesel"
29 },
30 {
31 "codigo": "2015-3",
32 "nome": "2015 Diesel"
33 },
34 {
35 "codigo": "2014-3",
36 "nome": "2014 Diesel"
37 },
38 {
39 "codigo": "2013-3",

Valor



Bônus

- Padrão **MVC** - Model View Controller
- O padrão MVC (Model-View-Controller) é um padrão arquitetural amplamente utilizado em desenvolvimento de software, especialmente em frameworks para construção de aplicativos web. Ele separa a aplicação em três componentes principais, cada um com responsabilidades específicas.

Bônus

- Padrão **MVC** - Model View Controller
- **Model (Modelo)**: Representa a camada de dados da aplicação. Responsável por gerenciar o estado da aplicação e realizar operações relacionadas aos dados. Geralmente, interage com o banco de dados ou outras fontes de dados.
- **Exemplo**: Usuario.java, Produto.java, etc.

Bônus

- Padrão **MVC** - Model View Controller
- **View (Visualização)**: Responsável pela apresentação dos dados ao usuário. Exibe as informações do modelo de maneira apropriada para o usuário. Geralmente, não realiza operações lógicas ou manipulações de dados, focando apenas na exibição.
- **Exemplo**: Tela de login (.html), de cadastro, etc.

Bônus

- Padrão **MVC** - Model View Controller
- **Controller (Controlador)**: Recebe e processa as entradas do usuário, como cliques de botões ou solicitações HTTP. Atua como um intermediário entre a camada de modelo e a camada de visualização. Responsável por atualizar o modelo com base nas interações do usuário e atualizar a visualização com base nas mudanças no modelo.
- **Exemplo**: LoginController.java, CarrinhoController.java, etc.

Bônus

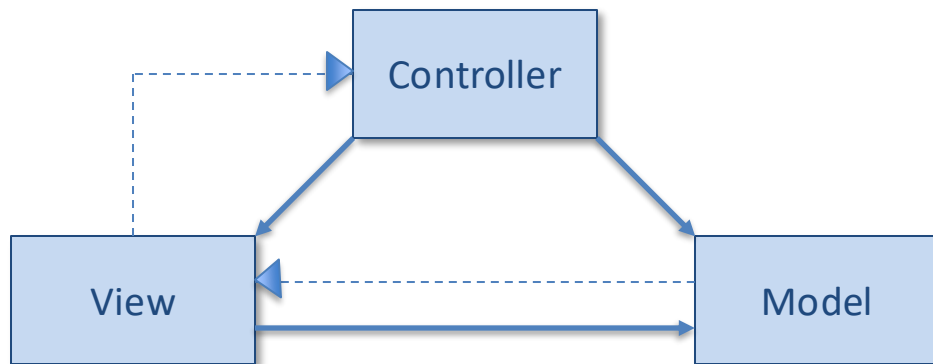
- Essa separação em três componentes distintos (Model, View e Controller) torna o código mais modular, facilitando a manutenção e a escalabilidade.
- Além disso, permite que diferentes partes da aplicação evoluam independentemente umas das outras.

Bônus

- Fluxo típico no padrão MVC:
 - 1) O usuário interage com a aplicação por meio da View.
 - 2) A View notifica o Controller sobre a interação do usuário.
 - 3) O Controller processa a interação e atualiza o Model conforme necessário.
 - 4) O Model notifica a View sobre as alterações nos dados.
 - 5) A View atualiza a interface do usuário para refletir as alterações no Model.

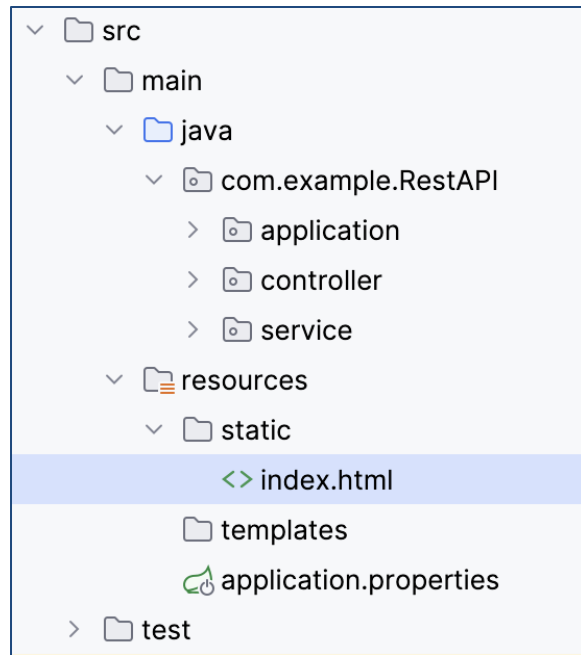
Bônus

- Fluxo típico no padrão MVC:



Bônus

- Esboçando um **front-end**:
 - Vamos inserir agora um **index.html** em uma camada **view** do nosso projeto.
 - No Spring Boot, a pasta **static** é um local especial para armazenar recursos estáticos, como arquivos HTML, CSS, JavaScript, imagens, etc.
 - Lembrando que esta é uma organização do Spring Boot. Em outros tipos de aplicações você poderia criar um mais pacote **view**, no mesmo nível dos pacotes **controller** e **model**.



Bônus

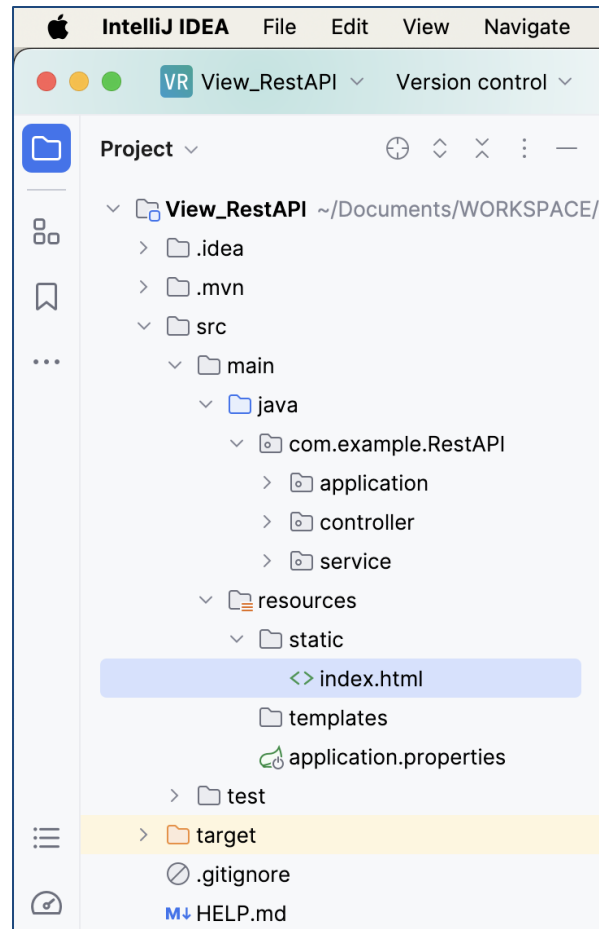
- Quando você coloca arquivos HTML nessa pasta, o Spring Boot automaticamente os trata como recursos estáticos e os disponibiliza diretamente para os clientes (navegadores) sem a necessidade de processamento adicional pelo controlador.
- A razão pela qual o Spring Boot utiliza a pasta static por padrão para recursos estáticos está relacionada à convenção sobre configuração.

Bônus

- O Spring Boot segue uma abordagem de "opinião sobre a configuração", o que significa que ele fornece configurações padrão sensíveis e automatizadas para ajudar os devs a começarem rapidamente, sem a necessidade de configurações extensivas.
- O objetivo aqui é que o desenvolvedor foque nas regras de negócio e deixe a configuração chata para o Spring Boot.

Bônus

- Esboçando um **front-end**:
 - Já a pasta **templates** no Spring Boot é um diretório especial designado para armazenar templates de visualização, geralmente usados em conjunto com engines de templates como Thymeleaf, FreeMarker ou Velocity.
 - O Spring Boot reconhece automaticamente os arquivos presentes nesta pasta e permite a fácil integração entre controladores e a camada de visualização.

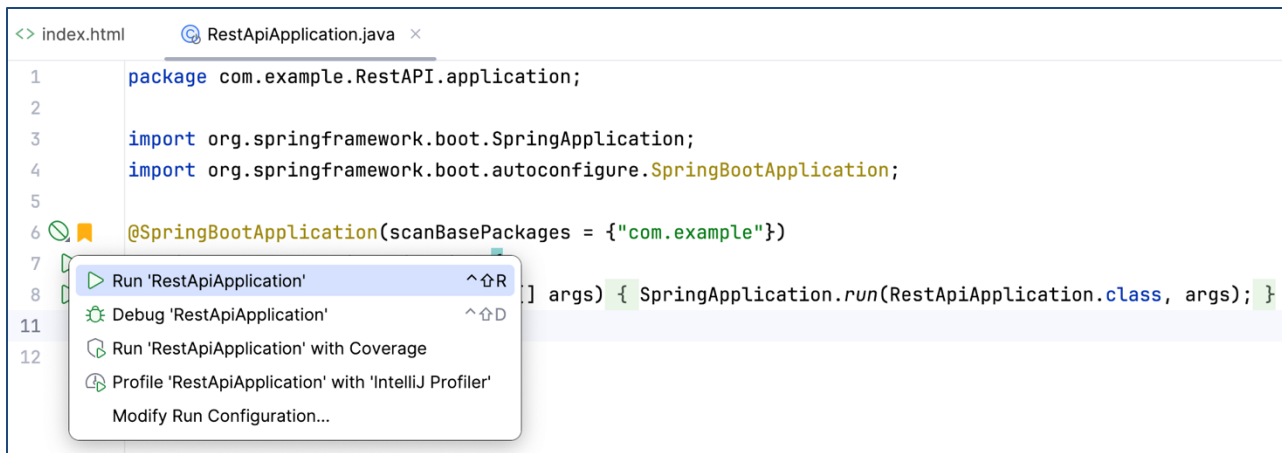


Bônus

- Adicione uma tag HTML qualquer ao arquivo **index.html** apenas para teste e, em seguida, execute a aplicação:



The screenshot shows a code editor window titled 'index.html'. The content of the file is a single line of HTML: `<h1>Hello World</h1>`. The text is highlighted in blue. A green checkmark is visible in the top right corner of the editor.



The screenshot shows a code editor window titled 'RestApiApplication.java'. The code is as follows:

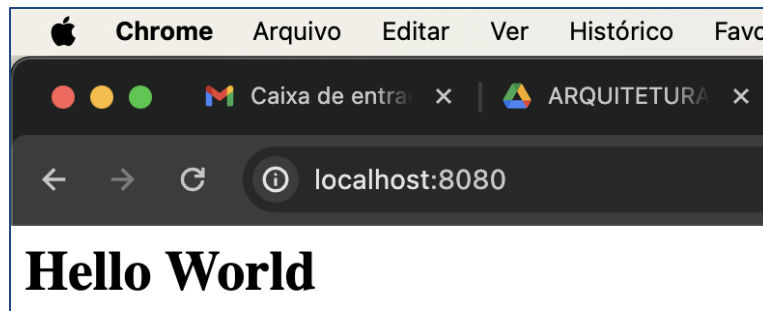
```
1 package com.example.RestAPI.application;  
2  
3 import org.springframework.boot.SpringApplication;  
4 import org.springframework.boot.autoconfigure.SpringBootApplication;  
5  
6 @SpringBootApplication(scanBasePackages = {"com.example"})  
7  
8 public class RestApiApplication {  
9     public static void main(String[] args) { SpringApplication.run(RestApiApplication.class, args); }  
10 }  
11  
12
```

A context menu is open over the code, showing the following options:

- Run 'RestApiApplication' (highlighted)
- Debug 'RestApiApplication'
- Run 'RestApiApplication' with Coverage
- Profile 'RestApiApplication' with 'IntelliJ Profiler'
- Modify Run Configuration...

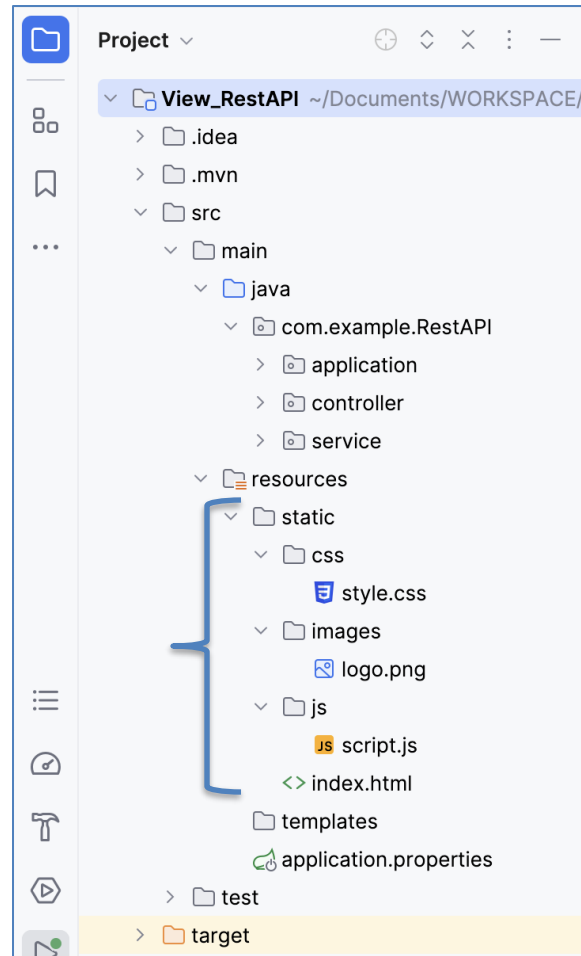
Bônus

- Por fim, abra seu navegador e acesse a URL:
 - <http://localhost:8080/>
- Veja que o Spring Boot automaticamente tratou o recurso estático (**index.html**) e o disponibilizou diretamente para o cliente (navegador) sem a necessidade de nenhum código adicional no controlador.

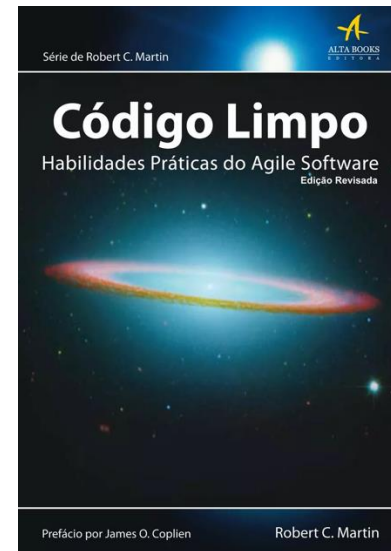
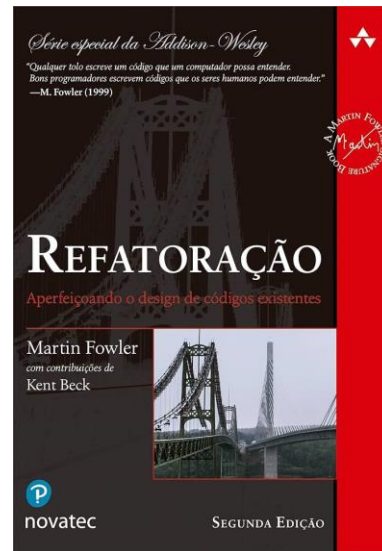
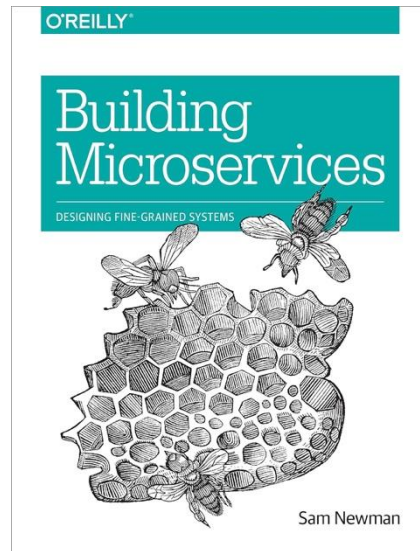


Bônus

- Esboçando um **front-end**:
 - Caso você queira adicionar uma folha de estilo **style.css**, um arquivo JavaScript **script.js** e uma imagem **logo.png**, esse seria um exemplo de estrutura da pasta **static**.
 - Em seguida, bastaria referenciar esses arquivos na página **index.html**.



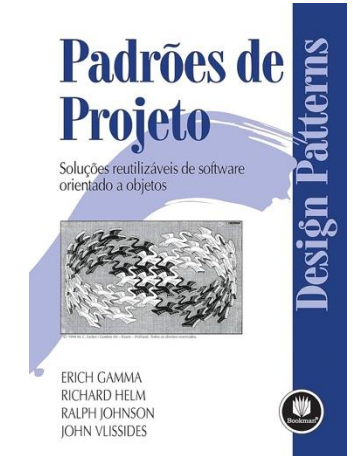
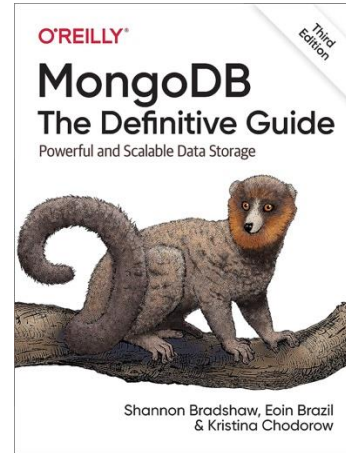
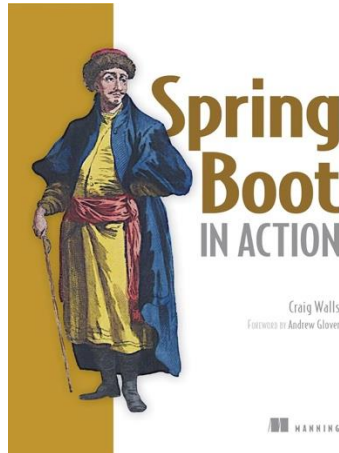
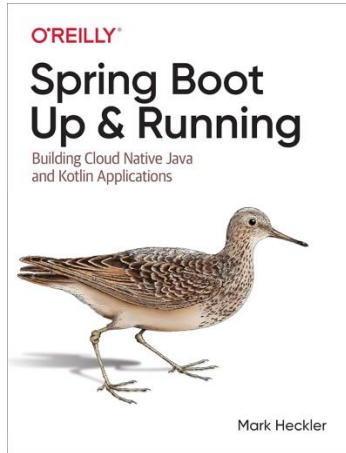
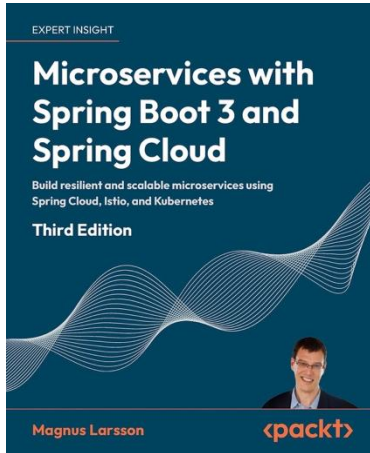
Referências básicas



Referências complementares



Outras referências



Obrigado!

Dúvidas?

joaopauloaramuni@gmail.com



[GitHub](#)



[LinkedIn](#)



[Lattes](#)

...